

UNIVERSIDAD SAN FRANCISCO DE QUITO USFQ

Colegio de Ciencias e Ingenierías

**Arquitectura “Mixture of Experts” para Clasificación de Ataques
en “Dataset” con Desbalance Extremo de Clases**

John Kléber Ochoa Abad

Ingeniería en Ciencias de la Computación

Trabajo de fin de carrera presentado como requisito
para la obtención del título de
Ingeniero en Ciencias de la Computación

Quito, 13 de mayo de 2026

UNIVERSIDAD SAN FRANCISCO DE QUITO USFQ

Colegio de Ciencias e Ingenierías

**HOJA DE CALIFICACIÓN
DE TRABAJO DE FIN DE CARRERA**

**Arquitectura “Mixture of Experts” para Clasificación de Ataques
en “Dataset” con Desbalance Extremo de Clases**

John Kléber Ochoa Abad

Nombre del profesor, Título académico

Ricardo Flores Moyano, PhD.

Quito, 08 de mayo de 2026

© DERECHOS DE AUTOR

Por medio del presente documento certifico que he leído todas las Políticas y Manuales de la Universidad San Francisco de Quito USFQ, incluyendo la Política de Propiedad Intelectual USFQ, y estoy de acuerdo con su contenido, por lo que los derechos de propiedad intelectual del presente trabajo quedan sujetos a lo dispuesto en esas Políticas.

Asimismo, autorizo a la USFQ para que realice la digitalización y publicación de este trabajo en el repositorio virtual, de conformidad a lo dispuesto en la Ley Orgánica de Educación Superior del Ecuador.

Nombres y apellidos: John Kléber Ochoa Abad

Código: 00345743

Cédula de identidad: 1719948216

Lugar y fecha: Quito, 04 de mayo de 2026

ACLARACIÓN PARA PUBLICACIÓN

Nota: El presente trabajo, en su totalidad o cualquiera de sus partes, no debe ser considerado como una publicación, incluso a pesar de estar disponible sin restricciones a través de un repositorio institucional. Esta declaración se alinea con las prácticas y recomendaciones presentadas por el Committee on Publication Ethics COPE descritas por Barbour et al. (2017) Discussion document on best practice for issues around theses publishing, disponible en <http://bit.ly/COPETheses>.

UNPUBLISHED DOCUMENT

Note: The following capstone project is available through Universidad San Francisco de Quito USFQ institutional repository. Nonetheless, this project – in whole or in part – should not be considered a publication. This statement follows the recommendations presented by the Committee on Publication Ethics COPE described by Barbour et al. (2017) Discussion document on best practice for issues around theses publishing available on <http://bit.ly/COPETheses>.

Resumen

El presente trabajo aborda la implementación de una arquitectura de Inteligencia Artificial tipo “Mixture of Experts” (MoE) para la detección y clasificación de intrusiones en ciberseguridad, un problema crítico en la protección de infraestructuras y sistemas. Utilizando el “Dataset” CIC-IDS2017, se enfrentó el desafío del desbalance extremo de clases, preservando la distribución estructural original con el fin de evitar la introducción de sesgos. La evaluación inicial, basada en el “F1-score” macro, evidenció que los modelos basados en árboles de decisión, como “Random Forest” (0.85) y “XGBoost” (0.83), superan a las redes neuronales estándar de tipo Perceptrón Multicapa (0.77) en la identificación de patrones. A partir de estos resultados, se determinó que la ponderación de clases mediante funciones de costo resulta más efectiva que técnicas como el sobremuestreo o la selección de características para mejorar el rendimiento de los modelos base. Con el objetivo de superar estos resultados, se seleccionó “Random Forest” como modelo base y se desarrollaron arquitecturas MoE incrementales, evolucionando hacia la integración de modelos homogéneos para expertos de tipo Red Neuronal “TabTransformer”. Los experimentos con MoEs demostraron que el uso de “Soft Gating” es superior al “Hard Gating”, ya que permite una combinación ponderada de expertos permitiendo capturar relaciones complejas mediante mecanismos de autoatención. Asimismo, se evidenció que la aplicación de tasas de aprendizaje independientes durante el proceso de “fine-tuning” es clave para lograr una adecuada sincronización del ensamble. Finalmente, el modelo MoE propuesto superó a los clasificadores tradicionales, alcanzando un “F1-score” macro de 0.89 y una precisión macro de 0.96. Estos resultados sugieren que la segmentación taxonómica de expertos y el enrutamiento continuo constituyen un enfoque efectivo y prometedor para el desarrollo de soluciones avanzadas de ciberseguridad orientadas a la protección de sistemas críticos mediante la distinción correcta de intrusiones respecto al tráfico benigno.

Palabras clave: “Mixture of Experts”, Inteligencia Artificial, Ciberseguridad, Ponderación por clases, Desbalance de clases, Sobremuestreo, Selección de Características, Funciones de Pérdida, Sistemas de detección de intrusiones, Redes Neuronales, “Random Forest”, “XGBoost”, “Tuning”

Abstract

This work addresses the implementation of Artificial Intelligence architecture based on Mixture of Experts (MoE) for the detection and classification of intrusions in cybersecurity, a critical problem in the protection of infrastructures and systems. Using the CIC-IDS2017 Dataset, the challenge of extreme class imbalance was faced, preserving the original structural distribution to avoid the introduction of biases. The initial evaluation, based on the F1-score macro, showed that models based on decision trees, such as Random Forest (0.85) and XGBoost (0.83), outperform standard Multilayer Perceptron neural networks (0.77) in pattern identification. From these results, it was determined that class weighting using cost functions is more effective than techniques such as oversampling or feature selection to improve the performance of base models. To overcome these results, Random Forest was selected as the base model and incremental MoE architectures were developed, evolving towards the integration of homogeneous models for experts of the TabTransformer Neural Network type. The experiments with MoEs showed that the use of soft gating is superior to hard gating, since it allows a weighted combination of experts allowing complex relationships to be captured through self-attention mechanisms. Likewise, it was evidenced that the application of independent learning rates during the fine-tuning process is key to achieve adequate assembly synchronization. Finally, the proposed MoE model outperformed traditional classifiers, achieving an F1-macro score of 0.89 and a macro accuracy of 0.96. These results suggest that taxonomic segmentation of experts and continuous routing constitute an effective and promising approach for the development of advanced cybersecurity solutions aimed at protecting critical systems through the correct distinction of intrusions from benign traffic.

Keywords: Mixture of Experts, Artificial Intelligence, Cybersecurity, Class weighting, Class imbalance, Oversampling, Feature Selection, Loss functions, Intrusion detection systems, Neural Networks, Random Forests, XGBoost, Tuning

Tabla de Contenido

Introducción	12
Estado del Arte.....	13
Desarrollo: Primera Parte: Data Engineering	14
Desarrollo: Segunda Parte: Modelos Base.....	20
Desarrollo: Tercera Parte: Mixture of Experts.....	35
Desarrollo: Cuarta Parte: Análisis de Resultados	47
Conclusiones	51
Trabajo Futuro	55
Referencias bibliográficas.....	57
Anexo A: Conteo Inicial de Muestras por Clase	59
Anexo B: Gráficos de Distribución de Features para tráfico Benigno y Ataques	60
Anexo C: Boxplots de Features por Clase para Flujos Benignos e Intrusiones.....	64
Anexo D: Tabla de Resumen Antes de la Limpieza de Datos	67
Anexo E: Tabla de Resumen Después de la Limpieza de Datos	68
Anexo F: Distribuciones por característica antes y después de imputaciones por regresores bayesianos y de tipo árbol de decisión.....	69

Índice de Tablas

Tabla 1. Distribuciones antes y después de la limpieza de datos.....	19
Tabla 2. Métricas de Mejores Modelos Random Forest por experimento.....	31
Tabla 3. Métricas de Mejores Modelos XGBoost por experimento	32
Tabla 4. Métricas de Mejores Modelos Red Neuronal de tipo Perceptrón Multicapa por experimento.....	33
Tabla 5. Métricas de Mejores Modelos MoE por arquitectura y sobremuestreo en subconjuntos de entrenamiento de expertos.....	46

Índice de Figuras

Figura 1. Distribución de Clases en “Dataset” de Intrusiones	15
Figura 2. Matriz de correlación para características del “Dataset” de Intrusiones	16
Figura 3. Pipeline para Limpieza de “Dataset” de Intrusiones	20
Figura 4. Pipeline para Generación de Modelos Base de Clasificación de Intrusiones.....	22
Figura 5. Arquitectura MoE “Hard Gating” Top-K con expertos Perceptrón Multicapa.....	39
Figura 6.Arquitectura MoE “Soft Gating” con expertos Perceptrón Multicapa	42
Figura 7. Arquitectura MoE “Soft Gating” con expertos TabTransformer	44

Introducción

El presente trabajo de fin de carrera consiste en el diseño, implementación y evaluación de una arquitectura de Inteligencia Artificial basada en “Mixture of Experts” para la detección y clasificación de intrusiones en sistemas. El tema surge de una cuestión clara: en situaciones reales, el tráfico de red benigno es inmensamente mayor que el malicioso. Este desbalance estadístico provoca que los modelos tradicionales se sesguen hacia la mayoría y fallen al detectar los ataques menos frecuentes [1]. Para mitigar este problema en etapas tempranas, la ingeniería de datos integra técnicas de preprocesamiento, tales como la generación de datos sintéticos [2]. Sin embargo, dado que estas estrategias a nivel de datos suelen ser insuficientes ante escenarios de desbalance extremo, el proyecto busca abordar el problema directamente desde la ingeniería del modelo. Para esto, se plantea una propuesta basada en una arquitectura “Mixture of Experts” que tenga el uso de múltiples expertos y un sistema de compuerta enfocado en potenciar el entrenamiento sobre clases minoritarias. Para validar el aporte real de esta propuesta, su desempeño se contrastará frente a modelos base tradicionales utilizando métricas robustas al desbalance, como el “Recall” por clase y el “F1-Score” macro para evidenciar la ganancia predictiva [3]. Este proyecto resulta fundamental en el contexto ecuatoriano y global, debido a que la acelerada digitalización incrementa cada día la exposición a ciberataques. Por ello, contar con modelos precisos de detección y prevención que protejan la infraestructura de un sistema y detecten amenazas inusuales sin dejarse sesgar por el tráfico normal, es una necesidad imperativa. Las siguientes secciones del documento detallarán la ingeniería de datos, la evaluación de los modelos base, la implementación técnica de arquitecturas “Mixture of Experts” y el análisis de los resultados obtenidos. A continuación, se presenta el estado del arte para posteriormente justificar las decisiones de diseño y destacar los aportes de este proyecto frente a otras alternativas actuales.

Estado del Arte

La detección y clasificación de intrusiones en sistemas de ciberseguridad mediante Inteligencia Artificial se abordó inicialmente mediante modelos tradicionales como “Support Vector Machine”, “Random Forest”, “XGBoost”. Si bien estos enfoques logran resultados aceptables, presentan limitaciones de generalización y escalabilidad [4]. Para superar esto, la investigación reciente ha mirado hacia el “Deep Learning” al emplear redes neuronales tipo MLP y de tipo convolucional [5]. Aunque con esto se logran mejoras en precisión, estos modelos activan todos sus parámetros en cada inferencia, elevando el costo computacional, y siguen siendo sensibles a clases poco representadas. Frente a esto, el proyecto busca aportar una arquitectura que delega la decisión a expertos condicionados a los inputs, reduciendo el cómputo innecesario. A la par, la literatura actual suele abordar el desbalance extremo en un “Dataset” de intrusiones mediante técnicas a nivel de datos como el sobremuestreo [6]. Aunque estas estrategias son beneficiosas, el aprendizaje del modelo sigue siendo global por lo que el modelo propuesto busca llenar este vacío mediante una arquitectura basada en que distintos expertos aprendan patrones de las clases minoritarias, sin depender de la manipulación previa de los datos. Esto se busca lograr mediante el paradigma “Mixture of Experts”, el cual ha sido validado en áreas como visión por computador y procesamiento de lenguaje natural, donde su principal ventaja es escalar la capacidad predictiva en escenarios de desbalance [7]. Y en el dominio de la detección de intrusiones, un estudio de la Universidad de Atenas, Grecia representa uno de los primeros esfuerzos en emplear el paradigma “Mixture of Experts” para clasificar tráfico en redes [8]. En dicha investigación, se demostró superioridad frente a clasificadores estándar, pero identificaron que el mecanismo de compuerta favorecía a las clases mayoritarias. Este proyecto extiende directamente esa línea de investigación,

proponiendo una arquitectura diseñada para el desbalance extremo, con el objetivo de garantizar la detección de ataques minoritarios y favorecer una generalización real.

Desarrollo: Primera Parte: Data Engineering

- **Definición del “Dataset”:**

En la fase inicial del proyecto, se seleccionó el conjunto de datos "Intrusion Detection Evaluation Dataset (CIC-IDS2017)", elaborado por la Universidad de New Brunswick [9]. Este “Dataset” incluye muestras de tráfico benigno y múltiples clases de ataques con datos extraídos de flujos de red mediante la herramienta “Cicflowmeter”. Los datos abarcan diversos protocolos de comunicación como HTTP, HTTPS, FTP, SSH y protocolos de correo electrónico a la vez que contienen datos de diferentes tipos de intrusiones como DDoS, Bot, Ataques Web, entre otros. Cabe destacar, que este tráfico fue capturado a partir de una topología de red realista que integra módems, “firewalls”, “switches”, “routers” y servidores operando bajo sistemas operativos heterogéneos [9]. Para la implementación de este proyecto, los archivos CSV correspondientes se descargaron y consolidaron en un “Dataframe”, estableciendo así la base operativa para la etapa de preprocesamiento y limpieza de datos, paso previo necesario para el entrenamiento tanto de los modelos base como de la arquitectura “Mixture of Experts”.

- **Caracterización del “Dataset”**

Como parte de la caracterización de los datos se identificaron los siguientes atributos:

- **Cantidad de Muestras:** 2830743
- **Cantidad de “Features”:** 78 con características primordiales para caracterizar tráfico como: flujo de bytes/s, flujo de paquetes/s, puerto de destino, duración del flujo, etc.

- **Cantidad de “Labels”:** 15 incluyendo la clase de tráfico benigno y 14 variaciones de intrusiones en sistemas de ciberseguridad

A la par, se buscó entender la distribución de las clases a lo largo de las muestras lo cual se muestra en la siguiente figura en la que evidentemente se observa el desbalance de clases:

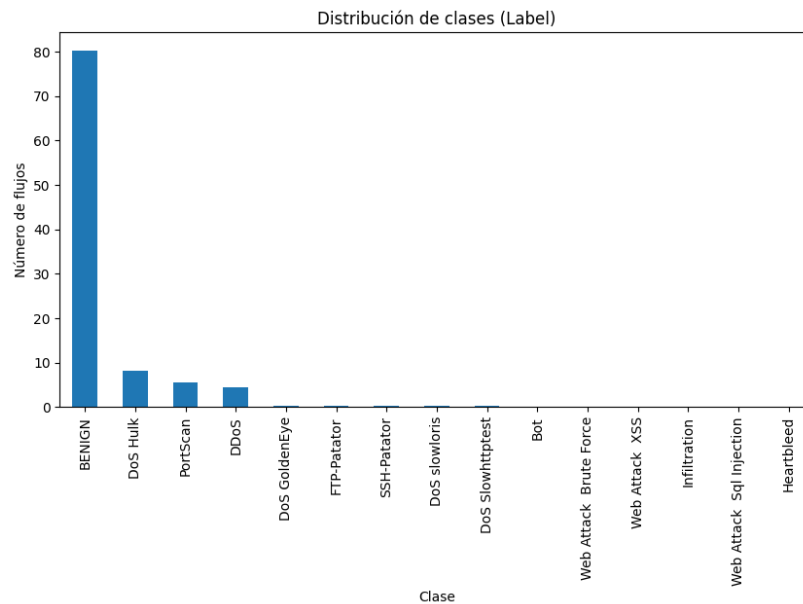


Figura 1. Distribución de Clases en “Dataset” de Intrusiones

Con base en dicha distribución, se buscó identificar el nivel de desbalance para lo cual se identificó que el 80,3% es tráfico benigno mientras que 19,7% corresponde a muestras de ataques de diferente tipo. A su vez, el “ratio” de desbalance corresponde a 206645.18 mientras que la entropía normalizada es de 0.2837 lo cual indica el desbalance extremo de clases. De igual forma, para entender con mayor detalle la cantidad de muestras por clase se puede verificar la tabla de conteo inicial de muestras por clase (Ver Anexo A) y gráficos de distribuciones y “boxplots” de características principales para caracterización de flujos de datos entre tráfico benigno e intrusiones (Ver Anexos B y C). Dicho análisis se fundamentó sobre la idea de encontrar características que en primera instancia permitan diferenciar el tráfico benigno de las intrusiones. La exploración de datos reveló que ciertas características permiten diferenciar claramente el tráfico benigno de las intrusiones. Variables relacionadas con la

longitud de los paquetes y los intervalos de tiempo entre paquetes, como la varianza de la longitud de los paquetes (16.23) y la desviación estándar del tiempo entre paquetes enviados (8.21), presentan diferencias significativas entre ambos tipos de tráfico. Además, medidas de inactividad como el tiempo mínimo, promedio y máximo de inactividad muestran patrones distintos, con valores de percentil 95 cercanos a 8.83×10^7 . Además, el máximo del tiempo entre paquetes del flujo, el máximo del tiempo entre paquetes enviados y el tamaño promedio de los paquetes destacan por su alta correlación e información mutua, mientras que características como las banderas urgentes de los paquetes y la longitud de los encabezados de los paquetes de regreso resultan poco relevantes. Estos hallazgos proporcionan criterios sólidos para seleccionar las variables más discriminativas en modelos de clasificación de intrusiones. Por último, a continuación, se agrega un gráfico de la correlación entre características el cual es importante para reconocer cuanto aportan las características a la distinción de flujos.



Figura 2. Matriz de correlación para características del “Dataset” de Intrusiones

- **Limpieza del “Dataset”**

El proceso de limpieza siguió buenas prácticas en detección de intrusiones, preservando la distribución original de las clases y eliminando inconsistencias. Para describir la estructura y calidad del “Dataset” previo al proceso de limpieza, se presenta una tabla resumen con estadísticas generales en el Anexo D y a continuación se describe paso a paso el proceso llevado a cabo:

1. Eliminación de columnas inconsistentes: Siguiendo las recomendaciones para la depuración de datos de tráfico de red [10], se identificaron variables con valores sin interpretación física válida, como magnitudes negativas. Se estableció como criterio formal de eliminación la presencia de valores inconsistentes en una proporción superior al 30% de las observaciones, dado que en estos casos la corrección individual puede introducir mayor sesgo que la exclusión de la variable [11]. Bajo este criterio, las columnas “Bytes de ventana inicial hacia atrás” y “Bytes de ventana inicial hacia adelante” fueron eliminadas. Asimismo, se descartó la característica duplicada “Longitud de cabecera hacia adelante”. Con estas acciones se redujo el número de características de 78 a 75.
2. Tratamiento de valores negativos remanentes: Para las columnas restantes con valores negativos se aplicó un criterio diferenciado según su distribución por clase. Ocho columnas presentaban valores negativos exclusivamente en la clase mayoritaria (tráfico benigno). En estos casos, se optó por la eliminación puntual de dichas instancias al representar menos del 0.007% de la clase, umbral estadísticamente despreciable ($<0.01\%$) a la par de que esto no generaba impacto sobre las muestras de las clases minoritarias [12].
3. Manejo de valores nulos iniciales: Se eliminaron 2,990 muestras pertenecientes a la clase mayoritaria que contenían valores nulos estructurales. Como criterio de decisión

se estableció que la eliminación sería aceptable si la variación máxima en proporciones de clase fuese inferior al 1%. La diferencia máxima observada fue de 2.08×10^{-4} , por lo que se consideró que la distribución no fue alterada significativamente [12]. Los valores nulos restantes se reservaron para imputación supervisada.

4. Eliminación de duplicados: En concordancia con la literatura sobre aprendizaje en escenarios desbalanceados [10], [13], se eliminaron registros duplicados exclusivamente en la clase mayoritaria. El criterio aplicado consistió en preservar íntegramente las clases minoritarias para evitar pérdida de información representativa, dado que la eliminación indiscriminada de duplicados en clases minoritarias puede incrementar artificialmente el desbalance relativo.
5. Eliminación de características constantes: Se identificaron ocho columnas con varianza nula. Conforme al criterio formal de selección de características, variables con varianza cero no aportan capacidad discriminativa ni información útil para modelos supervisados, por lo que fueron eliminadas, reduciendo el número de características de 75 a 67 [12].
6. Manejo de valores infinitos: Las variables “Paquetes por segundo en el flujo” y “Bytes por segundo en el flujo” presentaron valores infinitos derivados de divisiones entre duraciones extremadamente pequeñas. Siguiendo prácticas estándar de preprocesamiento numérico [14], estos valores fueron reemplazados por nulos, permitiendo su tratamiento posterior mediante técnicas de imputación supervisada.
7. Estado final del conjunto de datos: El conjunto resultante contiene 2 604 791 muestras y 67 características numéricas. La clase benigna representa el 78.59 % del total, con un fuerte desbalance (ratio 186 104 respecto a la clase más minoritaria).

El resumen de las métricas estadísticas posterior a la limpieza de datos se puede visualizar en el Anexo E. De dicho resumen se puede observar que los únicos nulos que quedan son los que

posteriormente se imputarían en el pipeline del entrenamiento de los modelos base, a su vez, se puede observar que no existen valores negativos en los mínimos ni valores infinitos en los máximos de cada muestra. A la par, con el objetivo de evaluar el impacto de limpieza sobre las variables del “Dataset”, se realizó un análisis comparativo de estadísticas descriptivas antes y después de la depuración. Para cada variable de las tablas presentadas en Anexos D y E se examinaron medidas de tendencia central, dispersión y rango intercuartílico. Los resultados como se puede verificar en dichas tablas muestran que las variaciones observadas fueron marginales, sin alteraciones sustanciales en la estructura estadística de las variables. De igual forma, se realizó un análisis comparativo de la distribución de clases antes y después del proceso de limpieza con el fin de verificar la preservación de la estructura original del “Dataset”. La reducción de muestras se concentró en la clase mayoritaria, presentando una variación máxima de 2.128 puntos porcentuales. Las clases minoritarias no fueron modificadas en términos absolutos, observándose únicamente incrementos relativos derivados de la reducción del total de muestras. En consecuencia, se concluye que la distribución del “Dataset” no fue alterada de manera significativa, manteniéndose el desbalance estructural original, en concordancia con las prácticas recomendadas para el tratamiento de datos desbalanceados [2].

label	Antes	Despues	Diferencia Absoluta	Diferencia Absoluta %
BENIGN	0.803003664	0.785915262	0.017088402	2.1280603
DoS Hulk	0.081629805	0.088710764	0.007080959	8.674477146
PortScan	0.05614427	0.061014492	0.004870222	8.674477146
DDoS	0.045227348	0.049150584	0.003923236	8.674477146
DoS GoldenEye	0.003636148	0.003951565	0.000315417	8.674477146
FTP-Patator	0.002804211	0.003047461	0.000243251	8.674477146
SSH-Patator	0.002083199	0.002263905	0.000180707	8.674477146
DoS slowloris	0.002047519	0.002225131	0.000177612	8.674477146
DoS Slowhttptest	0.0019426	0.00211111	0.00016851	8.674477146
Bot	0.000694517	0.000754763	6.02457E-05	8.674477146
Web Attack Brute Force	0.000532369	0.000578549	4.61802E-05	8.674477146
Web Attack XSS	0.000230328	0.000250308	1.99798E-05	8.674477146
Infiltration	1.27175E-05	1.38207E-05	1.10318E-06	8.674477146
Web Attack Sql Injection	7.41855E-06	8.06207E-06	6.4352E-07	8.674477146
Heartbleed	3.88591E-06	4.22299E-06	3.37082E-07	8.674477146

Tabla 1. Distribuciones antes y después de la limpieza de datos

Por último, el “pipeline” descrito para la limpieza de datos se resume en el siguiente diagrama:

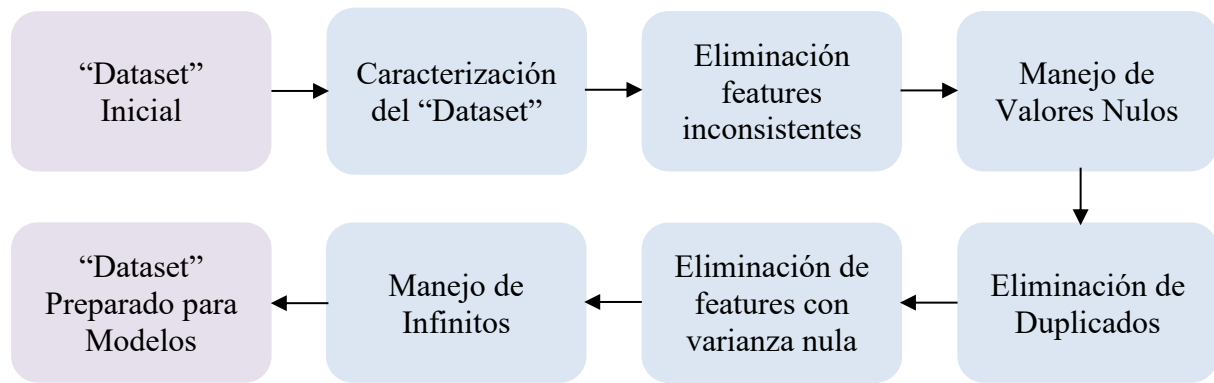


Figura 3. Pipeline para Limpieza de “Dataset” de Intrusiones

Desarrollo: Segunda Parte: Modelos Base

El objetivo de esta fase es establecer una línea base de rendimiento mediante la implementación de modelos de aprendizaje automático convencionales, categorizados como técnicas modernas para el manejo de datos tabulares de intrusiones acorde a la investigación previamente presentada de estado del arte. Estos modelos permitirán cuantificar la capacidad de discriminación inicial y servirán como punto de referencia a superar mediante la arquitectura de “Mixture of Experts”. Para este estudio, se seleccionaron tres algoritmos pilares: “Random Forest”, “XGBoost” y Redes Neuronales diseñadas para datos tabulares. La evaluación se centra en la métrica “F1-Score” Macro, seleccionada específicamente para mitigar el sesgo hacia las clases mayoritarias y garantizar que las clases minoritarias (intrusiones) posean el mismo peso en la evaluación del modelo. Adicionalmente, se analizarán el “Recall” por clase y las matrices de confusión para diagnosticar la correcta identificación de tráfico benigno y las intrusiones.

Para asegurar la reproducibilidad y la robustez de los modelos, se diseñó un flujo de trabajo estructurado de la siguiente manera:

1. **Partición de Datos:** División estratificada en conjuntos de entrenamiento, validación y prueba (70%, 15%, 15%) para preservar la distribución de clases en todos los subconjuntos/
2. **Codificación de Etiquetas:** Transformación de las etiquetas categóricas en valores numéricos, ajustando el codificador exclusivamente con los datos de entrenamiento.
3. **Normalización:** Aplicación de un escalado estándar para garantizar que las magnitudes de las características favorezcan la convergencia de los modelos basados en gradientes o distancias.
4. **Imputación por Clase:** Se implementó una estrategia de imputación por clase mediante regresores bayesianos y bosques aleatorios. Tras un análisis de preservación de la media y distribución, se seleccionó el regresor bayesiano por su estabilidad matemática. La comparativa de ambos imputadores se observa en el Anexo F.
5. **Sobremuestreo con Análisis de Artefactos (opcional):** Aplicación de técnicas de sobremuestreo sintético en set de entrenamiento. Se incluyó análisis de artefactos mediante vecinos cercanos para identificar generación de muestras sintéticas que puedan generar ruido.
6. **Selección de Características (opcional):** Uso de métodos supervisados basados en selección de características con base en los “labels” a nivel de cada clase para determinar la cantidad de características suficientes para maximizar la predictibilidad del modelo. Se decide no optar por extracción de características tipo PCA para evitar el riesgo de relacionar entre sí características que puedan aportar a distinción de clases minoritarias. También se incluyó en los experimentos la omisión de la selección de características.
7. **Optimización de parámetros de modelos:** Uso del “framework” Optuna para la búsqueda bayesiana de parámetros óptimos, maximizando el F1-Score Macro tras

múltiples experimentos con hiperparámetros acordes a cada uno de los tres tipos de modelos.

A continuación, se muestra gráficamente el pipeline previamente detallado para la generación de los modelos base:

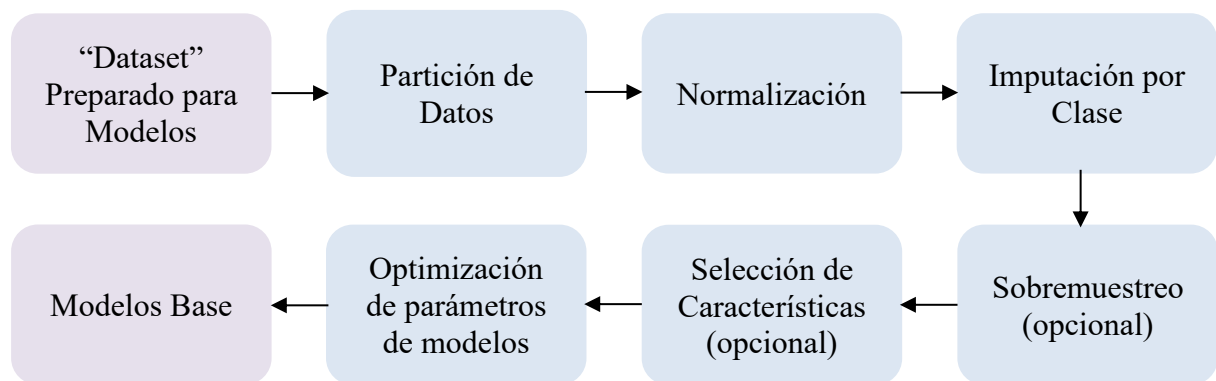


Figura 4. “Pipeline” para Generación de Modelos Base de Clasificación de Intrusiones

Dentro del “pipeline” descrito se ha implementado una serie de técnicas en los experimentos para mejorar el aprendizaje de los modelos. A continuación, se da un detalle de las técnicas fijas a implementar en los pipelines para mejorar el entrenamientos de los modelos base:

- **Imputación por Clase:** En ciberseguridad, los datos faltantes no suelen ser aleatorios. Imputar de forma general puede alterar la información de un ataque. En el presente caso, al usar regresores bayesianos por clase, el modelo estima los valores faltantes basándose en el comportamiento específico de cada tipo de tráfico, manteniendo la integridad de los datos de cada ataque [15]
- **Optimización con Optuna:** A diferencia de un Grid Search tradicional, Optuna emplea el algoritmo “Tree-structured Parzen Estimator”. Este método de optimización bayesiana permite explorar el espacio de hiperparámetros de forma inteligente,

priorizando regiones prometedoras y descartando configuraciones ineficientes mediante mecanismos de poda. Esta eficiencia es crítica ante la alta dimensionalidad del “Dataset” de intrusiones donde la generación de experimentos puede llegar a ser computacionalmente costosa [18].

A su vez, se seleccionaron técnicas las cuales se fueron valorando de manera individual y finalmente de manera conjunta en los experimentos llevados a cabo para cada uno de los algoritmos de clasificación seleccionados. El detalle de cada uno de los experimentos se describe posteriormente en el presente documento:

- **SMOTE, SMOTE-ENN y SMOTE-Tomek:** El uso de técnicas de sobremuestreo ha demostrado que aporta al aprendizaje de modelos con clases desbalanceadas. Para el presente caso se seleccionó el sobremuestreo simple de tipo SMOTE y se complementó el mismo con técnicas como SMOTE-ENN (“Edited Nearest Neighbors”) y “Tomek Links” las cuales tienen métodos de limpieza post muestreo, eliminando ejemplos que generan ambigüedad en las fronteras de decisión [16].
- **Pesos de Clase Dinámicos:** Debido a que algunas clases son extremadamente escasas, incluso el sobremuestreo puede no ser suficiente. Ante aquello, se implementaron funciones de costo para penalizar con mayor severidad los errores en clases raras. Para la selección de las funciones de pesos se llevó a cabo un análisis de distribución y de pesos para evitar asignaciones extremadamente bajas o altas. Tras dicho análisis se optaron para los experimentos por las funciones de tipo: “Effective Samples”, Polinomial, Focal y “Balanced” [17]. No se implementó pesos manuales por clases para generar “boosting” como se ha verificado que se ha implementado en otras investigaciones debido a que se considera que el mismo incluye un riesgo de introducir sesgo manual.

- **Selección de características por F-ANOVA y Árboles Clasificadores:** La selección de F-ANOVA y Árboles Clasificadores para selección de características permite evaluar dos enfoques algorítmicos eficientes e interpretables. Se eligió F-ANOVA como técnica de filtrado para modelar dependencias lineales mediante análisis de varianza. Y en contraparte, los Árboles Clasificadores se incluyeron como técnica integrada para capturar interacciones multivariadas y relaciones no lineales mediante la reducción de impureza. Se busca valorar ambas técnicas sobre los datos.
- **Función de pérdida de tipo “Focal Loss”:** En la clasificación de intrusiones, la abrumadora mayoría de tráfico normal genera ejemplos fáciles de predecir. La implementación de “Focal Loss” aborda este problema al reducir dinámicamente el peso asignado a estas instancias de alta confianza. Para el presente caso, el uso de esta función dirige el foco de atención del modelo hacia los ejemplos más difíciles de clasificar, como ataques sofisticados por lo que en los modelos que soportan funciones de pérdida como “XGBoost” o Redes Neuronales se buscó validar el aporte de la función “Focal Loss” [25].
- **Agrupación de Familias de Ataques Web:** Se observó a lo largo de los experimentos descritos que los ataques como “SQL Injection”, “Cross Site Scripting” y “Brute Force” comparten características de red similares a nivel de transporte por lo que los modelos se confundían entre dichas clases a la hora de intentar hacer una clasificación exacta. De modo que dentro de los experimentos se valoró el aporte a las métricas de agrupar labels de un mismo tipo de ataques.

A continuación, como parte de la presente sección se detallan los pormenores y resultados de los experimentos para los modelos base por cada uno de los tres tipos de algoritmos seleccionados para la clasificación. Cabe destacar que por cada experimento se generaron

logs incluyendo los datos a nivel de modelo y de igual forma se indican para cada experimento los mejores resultados obtenidos por cada una de las métricas de interés en lo relacionado a la clasificación de intrusiones:

- **“Random Forest”**

Se seleccionó el algoritmo de “Random Forest” para la clasificación de intrusiones debido a su alta capacidad para identificar características relevantes y modelar relaciones complejas en los datos. Al ser un método de “Ensemble Learning”, reduce el sobreajuste y mejora la generalización, además de ser robusto frente a un “Dataset” desbalanceado, favoreciendo la detección de clases minoritarias. Asimismo, permite estimar la importancia de las variables y mantiene un buen equilibrio entre rendimiento y eficiencia computacional, lo que lo hace adecuado para sistemas de detección en tiempo real. Como referencia, el estudio de Chavan y Alone [19], utilizando el “Dataset” CIC-IDS2017 y técnicas de mejora tanto del lado del modelo como en el procesamiento de los datos obtuvo un desempeño destacado del modelo, alcanzando un “F1-score” macro de 96.7%, junto con altos valores de precisión y “recall”. Este resultado evidencia la efectividad de “Random Forest” para detectar múltiples tipos de ataques en escenarios reales.

Para todas las iteraciones, se utilizó Optuna como motor de optimización bayesiana bajo las siguientes condiciones constantes:

- Métrica Objetivo: Maximización del “F1-Score” Macro en el set de validación.
- Validación: División estratificada (Benignos vs. Ataques) para asegurar representatividad.

Espacio de Búsqueda de Hiperparámetros:

- `n_estimators`: [50, 400]
- `max_depth`: [10, 60]
- `min_samples_split`: [2, 15]
- `max_features`: ['sqrt', 'log2', None]

- **“XGBoost”:**

Se seleccionó el algoritmo “XGBoost” para la clasificación de intrusiones debido a su alta eficiencia y capacidad para modelar relaciones complejas no lineales mediante la construcción secuencial de árboles de decisión. Al ser un algoritmo basado en “Gradient Boosting”, penaliza los errores de los árboles anteriores, mejorando continuamente la clasificación de las instancias más difíciles respecto al uso de “Random Forest”. Además, incluye técnicas de regularización integradas que previenen el sobreajuste y aprovecha la aceleración por hardware, lo que permite procesar grandes volúmenes de datos con una alta eficiencia computacional [20].

Como referencia, el estudio de Sow y Ada [21], utilizando de igual forma el “Dataset” CIDS 2017, reporta un desempeño destacado del modelo XGBoost, alcanzando una precisión macro de 99%. Este resultado evidencia la efectividad del modelo para minimizar falsos negativos en escenarios de detección de intrusiones.

Para todas las iteraciones, se utilizó Optuna como motor de optimización bayesiana bajo las siguientes condiciones constantes:

- Métrica Objetivo: Maximización del F1-Score Macro en el set de validación.
- Validación: División estratificada (Benignos vs. Ataques) para asegurar representatividad.

Espacio de Búsqueda de Hiperparámetros:

- `n_estimators`: [150, 450]
- `max_depth`: [6, 15]
- `learning_rate`: [0.01, 0.2] (escala logarítmica)
- `subsample`: [0.6, 1.0]
- `colsample_bytree`: [0.6, 1.0]

- **Redes Neuronales**

Se seleccionó el uso de Redes Neuronales de tipo Perceptrón Multicapa (MLP) debido a su capacidad superior para extraer representaciones jerárquicas y patrones altamente complejos en el tráfico de red. A diferencia de los modelos basados en árboles, las redes neuronales pueden escalar masivamente con los datos y beneficiarse de la aceleración por hardware [5].

Como referencia, investigaciones recientes en el dominio de detección de intrusiones [5] destacan que las arquitecturas optimizadas logran reducir significativamente las tasas de falsos positivos en conjuntos de datos complejos como CIC-IDS2017, justificando su implementación para capturar intrusiones que métodos tradicionales podrían omitir.

Se continuó utilizando Optuna como motor de optimización bajo las siguientes condiciones constantes:

- Métrica Objetivo: Maximización del F1-Score Macro en el set de validación.
- Validación: División estratificada para asegurar representatividad de las clases.

Espacio de Búsqueda de Hiperparámetros:

- Tamaño de lote: [4096, 8192, 16384]

- Número de capas: [2, 10]
- Neuronas por capa: [512, 1024]
- Tasa de aprendizaje: [1e-4, 1e-3] (escala logarítmica)
- Número de épocas: [50,100]

- **Descripción de Experimentos para los algoritmos**

Atendiendo a las mejores prácticas de experimentación en aprendizaje automático, se estructuró una metodología basada en bloques controlados. Esta aproximación como se ha evidenciado en la investigación de mejoras metodológicas de Saini et. al. [22] permite aislar el impacto de cada técnica garantizando una interpretación correcta de los resultados y evitando el enmascaramiento de las métricas:

- **Experimento 1 - Definición de Línea Base Comparativa:** Se estableció un punto de base evaluando únicamente el modelo sobre el “Dataset” original. En esta fase no se aplicó ninguna técnica de sobremuestreo, asignación de pesos dinámicos, reducción de dimensionalidad, ni agrupación de clases. Este modelo servirá como control absoluto para medir la ganancia o pérdida real de rendimiento introducida por las técnicas posteriores y a su vez funcionará como punto de comparación respecto a los modelos “Mixture of Experts” para comparar modelos puros entre sí [23].
- **Experimento 2 - Evaluación Aislada de Técnicas de Sobremuestreo:** Partiendo del modelo base, se evaluó de manera individual el impacto de la generación de datos sintéticos. Se ejecutaron pruebas separadas para SMOTE simple, SMOTE-ENN y SMOTE-Tomek [6]. El objetivo de este bloque fue identificar exclusivamente cómo reacciona el modelo al balanceo de clases orientado a aumentar las muestras de clases minoritarias para mejorar el aprendizaje.

- **Experimento 3 - Evaluación Aislada de Pesos de Clase Dinámicos:** Volviendo al “Dataset” original, se integraron de forma independiente las funciones de penalización de costos: “Effective Samples”, Polinomial, Focal y “Balanced”. Este bloque controló cómo el modelo maneja el desbalanceo severo únicamente a través del peso de los errores en el proceso de aprendizaje [16].
- **Experimento 4 - Evaluación Aislada de Selección de Características:** De manera independiente al balanceo y a los pesos, se aplicaron técnicas de selección de variables sobre el modelo base. Se contrastó el rendimiento utilizando un enfoque de filtrado (F-ANOVA) frente a un enfoque integrado (Árboles Clasificadores). Esto permitió aislar y valorar el aporte específico de la selección de características siguiendo la recomendación de una investigación citada del Instituto de Tecnología de Indonesia respecto a CIDS [24]
- **Experimento 5 - Evaluación Aislada de Funciones de Pérdida:** En los modelos basados en optimización de gradiente como XGBoost o Redes Neuronales, se evaluó de forma aislada el impacto de la función “Focal Loss”. Se partió de un modelo base configurado con la función estándar de Entropía Cruzada como punto de control [25]. El objetivo de esta comparación directa fue medir exclusivamente si penalizar los ejemplos fáciles de la clase mayoritaria logra enfocar el aprendizaje hacia los ataques minoritarios difíciles de clasificar.
- **Experimento 6 - Integración Conjunta de Mejores Técnicas:** Una vez identificados los enfoques individuales con mejor rendimiento en los experimentos 2, 3, 4 y 5, se procedió a combinarlos. Este experimento integró la mejor técnica de sobremuestreo, la mejor función de peso dinámico, el mejor selector de características y la mejor función de pérdida.

- Experimento 7 - Análisis de Impacto por Agrupación de Clases:** Como estudio adicional, se evaluó el impacto directo de agrupar las clases minoritarias de ataque Web (“SQL Injection”, “Cross Site Scripting” y “Brute Force”) bajo un único “label”. Se comparó el rendimiento de este enfoque frente a la línea base comparativa del problema multiclase original de forma completamente aislada. El objetivo de este experimento fue justificar empíricamente si esta agrupación aporta a las métricas objetivo de la clasificación considerando que los ataques web tienen misma firma de características de tráfico independientemente del tipo de técnica de intrusión aplicada a nivel de capa de aplicación [26]

- Mejores Resultados de Modelos Base en Set de Pruebas**

- “Random Forest”**

Experimento	Técnica	F1 Macro	Recall Macro	Precisión Macro	Observación Principal
1 (base)	Ninguna	0.85	0.85	0.86	Alta profundidad y número de estimadores, junto con el uso completo de características
2	SMOTE_ENN	0.84	0.87	0.84	SMOTE-ENN, alta profundidad y división de hojas más restrictivo
3	Balanced	0.87	0.88	0.87	Pesos “balanced” mejora F1-macro, con profundidad moderada
4	Tree	0.85	0.85	0.87	La selección de características mantiene rendimiento, apoyada por alta profundidad

5 (no aplica)					
6	Solo Pesos Balanced	0.86	0.89	0.84	Los mejores resultados se obtuvieron con pesos “Balanced”
7	Agrupar clases Web	0.93	0.91	0.98	Agrupar clases web más modelo de alta complejidad y uso total de características

Tabla 2. Métricas de Mejores Modelos Random Forest por experimento

- “XGBoost”

Experimento	Técnica	F1 Macro	Recall Macro	Precisión Macro	Observación Principal
1 (base)	Ninguna	0.83	0.82	0.86	Profundidad y número de estimadores altos junto con tasa de muestreo parcial
2	SMOTE TOMEK	0.83	0.85	0.83	Con SMOTE-Tomek más el uso de muestreo parcial y una tasa de aprendizaje baja
3	Balanced	0.86	0.87	0.85	Pesos “Balanced” apoyados por un alto número de estimadores
4	F-ANOVA	0.83	0.81	0.85	Selección de características reduce rendimiento, aunque es estable con muestreo parcial
5	“Focal Loss”	0.83	0.82	0.86	Se observa degradación con “Focal Loss” en “XGBoost”, pese a muestreo casi completo

6	Solo Pesos Balanced	0.87	0.89	0.85	Pesos “Balanced” impulsa el rendimiento, con profundidad moderada y muestreo parcial
7	Agrupar clases Web	0.90	0.87	0.98	Agrupación de clases logra un “F1-macro” casi perfecto incluso con baja profundidad.

Tabla 3. Métricas de Mejores Modelos XGBoost por experimento

- **Redes Neuronales tipo Perceptrón Multicapa**

Experimento	Técnica	F1 Macro	Recall Macro	Precisión Macro	Observación Principal
1	Ninguna	0.77	0.75	0.92	Arquitectura poco profunda tipo bottleneck con alto tamaño de lote
2	SMOTE TOMEK	0.75	0.82	0.79	Con SMOTE-Tomek se mantiene un rendimiento limitado
3	Polinomial	0.73	0.72	0.85	La ponderación polinómica aporta una mejora apoyada por mayor profundidad
4	Tree	0.73	0.71	0.92	La selección no aporta mejoras, manteniendo un rendimiento similar al base
5	Focal Loss	0.75	0.73	0.91	Focal Loss genera una leve degradación en las métricas generales

6	Focal Loss + SMOTE TOMEK	0.76	0.83	0.80	Combinación de técnicas aporta una mejora con una arquitectura simple
7	Agrupar clases Web	0.79	0.89	0.75	Agrupación de clases impulsa el rendimiento acompañado de una mayor regularización

Tabla 4. Métricas de Mejores Modelos Red Neuronal de tipo Perceptrón Multicapa por experimento

Como complemento a los resultados mostrados en las tablas superiores a nivel de cada uno de los modelos base, se puede observar del lado del Anexo G las respectivas matrices de confusión para obtener un detalle más exacto de la identificación cuantitativa a nivel de la distinción de intrusiones respecto a tráfico benigno.

- **Discusión de Resultados**

Discusión de Resultados de los Modelos Base

- Los experimentos evidencian que los modelos basados en árboles de decisión superan consistentemente a las Redes Neuronales estándar para este conjunto de datos. “Random Forest” se posiciona como el modelo base más sólido, logrando un “F1-Macro” de 0.85 sin ninguna técnica adicional, mientras que la Red Neuronal base se estanca en un 0.77. Esto confirma la dificultad inherente de los perceptrones multicapa (MLP) tradicionales para modelar este tipo de datos tabulares sin arquitecturas especializadas [27].

- En las tres arquitecturas evaluadas, el ajuste de resultó ser una estrategia considerablemente más efectiva y estable que el sobremuestreo con técnicas como SMOTE_ENN o SMOTE TOMEK. En “Random Forest”, el uso de pesos incrementó el “F1 Macro” a 0.87 y el “Recall” Macro a niveles de 0.88. Por el contrario, SMOTE tendió a degradar ligeramente el F1 Macro o mantenerlo estático, sugiriendo que la introducción de ruido sintético no favorece las fronteras de decisión complejas de estos ataques [16].
- El experimento 7 ("Agrupar clases Web") generó el incremento de rendimiento más drástico en todos los modelos. En “Random Forest”, esta técnica disparó el F1 Macro a 0.93 y la precisión a 0.98. En “XGBoost” el F1 Macro subió a 0.90, y en la Red Neuronal a 0.79. Esto indica que existe un alto grado de solapamiento en las características de las subclases de ataques Web; al unificarlas, se reduce el ruido de clasificación y los algoritmos logran mejores métricas.
- La implementación de Focal Loss en la Red Neuronal tuvo un impacto sobre las métricas generales validando de esta manera que la misma no aportó en el presente caso a favorecer a la clasificación de clases minoritarias ya que en dicho proceso termino generando un impacto general respecto al modelo base.
- Las técnicas de selección y transformación de características no aportaron mejoras. En el caso de las Redes Neuronales, redujeron el F1 Macro a 0.73, y en XGBoost lo mantuvieron en 0.83 con caídas en el Recall. Esto sugiere que prescindir de características originales penaliza la detección de ataques con firmas sutiles.
- La brecha de rendimiento entre el Random Forest optimizado y la mejor Red Neuronal fundamenta técnicamente los desarrollos posteriores del proyecto. Demuestra que para que el aprendizaje profundo sea competitivo en este dominio, no basta con ajustes de hiperparámetros o funciones de pérdida en un MLP simple, sino que es estrictamente

necesario implementar arquitecturas que capturen relaciones más complejas, como el ensamble dinámico propuesto en el “Mixture of Experts” (MoE) o los mecanismos de un modelo de red neuronal más orientado a datos tabulares [8].

Desarrollo: Tercera Parte: Mixture of Experts

Un modelo “Mixture of Experts” (MoE) consiste en una arquitectura compuesta por submodelos especializados, denominados expertos, y un modelo integrador de tipo compuerta. Los expertos se especializan en clasificar subconjuntos específicos de datos, mientras que la compuerta aprende a ponderar y seleccionar las mejores subredes para cada instancia. La clasificación final se genera mediante la combinación de las probabilidades de salida de cada experto y los pesos asignados por la compuerta [28]. En contextos como el del presente proyecto con conjuntos de datos severamente desbalanceados, ciertas categorías presentan una mayor complejidad de clasificación. Esto se debe a la escasez de muestras para definir fronteras de decisión adecuadas o a la dificultad inherente de identificar ataques sofisticados. Para abordar estas limitaciones, que suelen afectar a los modelos de entrenamiento general, se justifica la implementación de una arquitectura MoE que fomente la especialización en vectores de ataque específicos con el fin de incrementar el rendimiento predictivo para lo cual en la presente sección se busca emplear este tipo de modelo para lograr una métrica “F1-score” macro superior al modelo base definido en la sección previa [8]. Por recordar, dicho modelo base consiste en el algoritmo tipo “Random Forest” sin aplicación de técnicas adicionales de mejora de modelos (ej. sobremuestreo, selección de características, ponderación de pesos) a manera de realizar una comparación directa de modelos. A continuación, se describen los detalles del modelo base Random Forest tomado para la comparativa base:

- **Modelo Base:**
- Tipo de Modelo: Random Forest
- Número de estimadores: 314
- Profundidad máxima: 83
- Muestras mínimas para división: 5
- Muestras mínimas por hoja: 2
- Características máximas: Ninguno
- Criterio de división: Entropía
- **Métricas Modelo Base:**
- F1-macro: 0.85
- Precisión macro: 0.85
- Recall macro: 0.86

Con base en dicho modelo el objetivo será mejorar la métrica de “F1-score macro” y para aquello, se implementaron arquitecturas MoE incrementales las cuales se detallan a continuación:

1. Arquitectura de tipo “Hard Gating” Top-K con expertos tipo Perceptrón Multicapa

A continuación, se detalla la configuración de la primera arquitectura implementada:

- Estrategia de Partición y Balanceo Específico: El conjunto de entrenamiento se segmentó en seis subconjuntos especializados, mapeando las clases originales hacia grupos lógicos de ataques. Para contrarrestar el desbalanceo severo, se integró una tubería de sobremuestreo dinámico, evaluando el impacto de técnicas como SMOTE, SMOTE-Tomek y SMOTE-ENN sobre el espacio de características previo a la

inyección de muestras sintéticas. Esto se debe a que, en arquitecturas Mixture of Experts con desbalance de clases, es recomendable aplicar técnicas de sobremuestreo a nivel de los subconjuntos de entrenamiento de cada experto, con el fin de garantizar una cantidad suficiente de muestras para el adecuado entrenamiento de los modelos especializados. La conformación de los expertos se definió siguiendo lineamientos de investigaciones previas, organizándolos según el volumen de ataques (de mayor a menor) y considerando familias de ataques de difícil clasificación, a partir de las cuales se generaron subconjuntos de datos específicos para cada experto [29].

- Experto 1: Tráfico Benigno.
 - Experto 2: Ataques Mayoritarios (DDoS, DoS Hulk, PortScan).
 - Experto 3: Ataques de Frecuencia Media (DoS GoldenEye, DoS Slowhttptest, DoS slowloris, FTP-Patator, SSH-Patator).
 - Experto 4: Ataques Botnet.
 - Experto 5: Ataques Web (Brute Force, SQL Injection, XSS).
 - Experto 6: Ataques Minoritarios (Heartbleed, Infiltration).
-
- Diseño y Pre-entrenamiento Individual de Expertos: Cada experto se modeló como un Perceptrón Multicapa adaptado a datos tabulares, incorporando capas de “Batch Normalization”, “Dropout” y funciones de activación. La topología de la red (profundidad y unidades por capa) y la tasa de aprendizaje se optimizaron de forma independiente para cada experto mediante Optuna y se empleó “Focal Loss” como función de costo para penalizar los errores en las muestras de mayor dificultad dentro de cada subconjunto [25]. Se seleccionó una Red Neuronal tipo MLP para los expertos del modelo “Mixture of Experts” (MoE), aunque no fuera el mejor modelo base, debido a su capacidad de optimización conjunta con el modelo de enrutamiento mediante el

ajuste simultáneo de pesos. A diferencia de modelos como “Random Forest”, esta propiedad permite una mejor adaptación e interacción entre expertos y enrutador dentro de la arquitectura MoE.

- **Mecanismo de Enrutamiento:** El orquestador del ensamble se construyó mediante una red neuronal paralela igual de tipo MLP con una capa de salida “Softmax”, encargada de emitir los pesos de contribución de cada experto [28]. Este mecanismo se encarga de evaluar el vector de características de entrada y calcula la probabilidad condicional de competencia de cada experto para la clasificación del flujo de red correspondiente.
- **Enrutamiento Estricto (Top-K):** Posterior a la validación del enrutamiento probabilístico continuo, se integró una variante de decisión estricta de tipo Top-K. Este esquema restringe la salida final del MoE a las predicciones de los K expertos con mayor probabilidad por muestra, aislando el resto mediante una máscara de ceros con el objetivo de mitigar el ruido predictivo. Dicho valor K se optimizó de igual forma con el uso de Optuna [30].
- **Pipeline de Entrenamiento por Fases:** La consolidación de la arquitectura se estructuró en una secuencia de acoplamiento de tres etapas de entrenamiento para garantizar la convergencia. Esta estructura de aprendizaje se considera “State of the Art” ya que permite tener un mejor control sobre el proceso progresivo de mejora del modelo en el entendimiento de patrones para clasificaciones [28]. A continuación, se detalla la estructura del mismo empleada en la primera arquitectura:
 - **Pre-entrenamiento:** Entrenamiento y almacenamiento independiente de los seis expertos.
 - **Entrenamiento de Compuerta:** Instanciación del MoE congelando los gradientes de los expertos para entrenar exclusivamente la red de enrutamiento.

- Refinamiento Conjunto: Descongelamiento de todos los parámetros globales y ejecución de una fase de ajuste fino con una tasa de aprendizaje reducida, preservando la especialización previa de los expertos.

Diagrama:

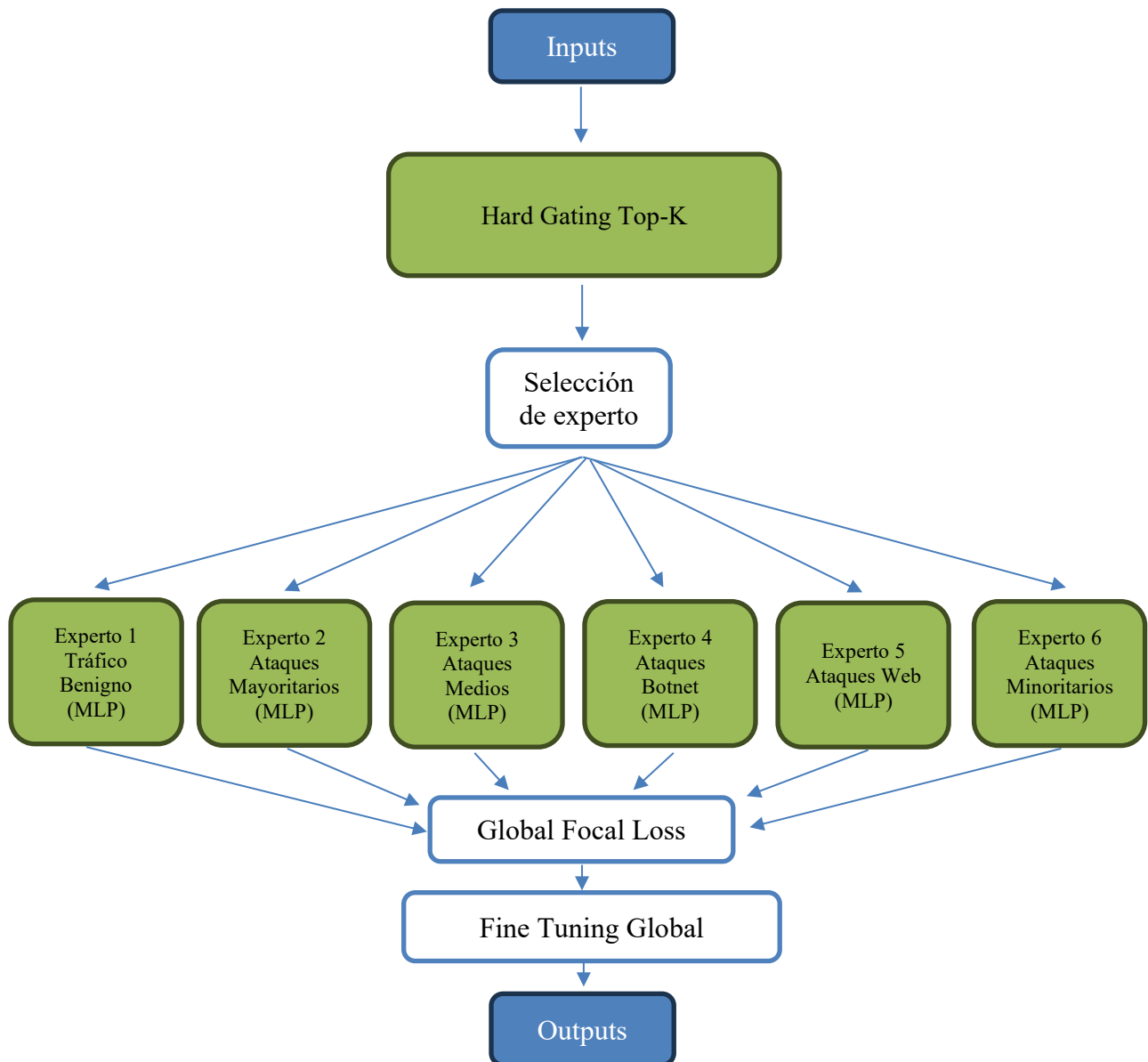


Figura 5. Arquitectura MoE "Hard Gating" Top-K con expertos Perceptrón Multicapa

2. Arquitectura de tipo “Soft Gating” con expertos tipo Perceptrón Multicapa

A continuación, se detalla la configuración de la segunda arquitectura implementada:

- Estrategia de Partición y Balanceo Específico: En esta segunda iteración, se reestructuró la taxonomía del ensamble, reduciendo la partición a cinco subconjuntos especializados. Esta agrupación se fundamentó en el análisis de las matrices de confusión previas, con el objetivo de consolidar clases de ataques que exhiben patrones similares y resultan difíciles de distinguir por un modelo general, aprovechando así la especialización de los expertos. Esta técnica se implementó como aporte propio considerando que se requería mejorar la clasificación focalizando el entrenamiento entre expertos que tienen fronteras de decisión compartidas.

Se optimizó la detección agrupando el tráfico benigno con aquellos ataques que tienden a camuflarse en el flujo normal, y se dividieron las familias de denegación de servicio en categorías más granulares. Se conservó el “pipeline” de sobremuestreo dinámico, evaluando las técnicas SMOTE, SMOTE-Tomek y SMOTE-ENN. La nueva distribución de expertos se definió de la siguiente manera:

- Experto 1: Tráfico Benigno y Ataques Camuflados (tráfico con patrones similares al no malicioso: Benigno, Bot, “Brute Force”, “SQL Injection”, “Cross Site Scripting”).
- Experto 2: Ataques Mayoritarios (DDoS, DoS Hulk, “PortScan”).
- Experto 3: Ataques DoS (“DoS GoldenEye”, “DoS Slowloris”, “DoS Slowhttpstest”).
- Experto 4: Ataques Patator (“FTP-Patator”, “SSH-Patator”).
- Experto 5: Ataques Minoritarios (Heartbleed, “Infiltration”).

- **Diseño y Preentrenamiento Individual de Expertos (TabularMLP):** Se mantuvo el diseño base, modelando cada experto como un Perceptrón Multicapa (MLP) adaptado a datos tabulares, integrando capas de “Batch Normalization”, “Dropout” y funciones de activación. La arquitectura topológica y los hiperparámetros fueron optimizados dinámicamente de forma individual mediante Optuna, empleando “Focal Loss” como función de costo debido a sus reconocidos beneficios en investigaciones de clasificación de intrusiones al apoyar el aprendizaje de clases minoritarias [25].
- **Mecanismo de Enrutamiento (“Soft Gating”):** En esta arquitectura, el enrutador opera como un coordinador que analiza los vectores de características de entrada y determina la idoneidad de cada experto, asignándoles una probabilidad mediante una capa “Softmax”. Esta estrategia de enrutamiento suave permite que todos los expertos contribuyan simultáneamente a la predicción final; sin embargo, el nivel de influencia de cada uno está estrictamente ponderado por el coeficiente de confianza emitido por la red de enrutamiento [31].
- **Pipeline de Entrenamiento y Refinamiento Diferencial:** Para favorecer la convergencia del ensamble bajo el esquema de “Soft Gating”, se optimizó el proceso de entrenamiento implementando una actualización simultánea con tasas de aprendizaje diferenciales. Se permitió el ajuste concurrente de toda la arquitectura (enrutador y expertos) sin congelar gradientes. Para lograrlo, se asignó una tasa de aprendizaje superior a la red de enrutamiento para acelerar su capacidad de delegación, mientras que a los expertos se les aplicó una tasa reducida para evitar la alteración abrupta de los pesos adquiridos durante el preentrenamiento. Este acoplamiento dinámico permitió la sinergia del sistema sin comprometer la especialización previa de las subredes [28].

Como se evidenciará en la sección de resultados, esta iteración arquitectónica logró superar el rendimiento del modelo base, consolidándose como el punto de partida óptimo para refinamientos posteriores en la aplicación de “Mixture of Experts” para la clasificación de intrusiones. A continuación, se presenta el diagrama esquemático de la arquitectura descrita.

Diagrama:

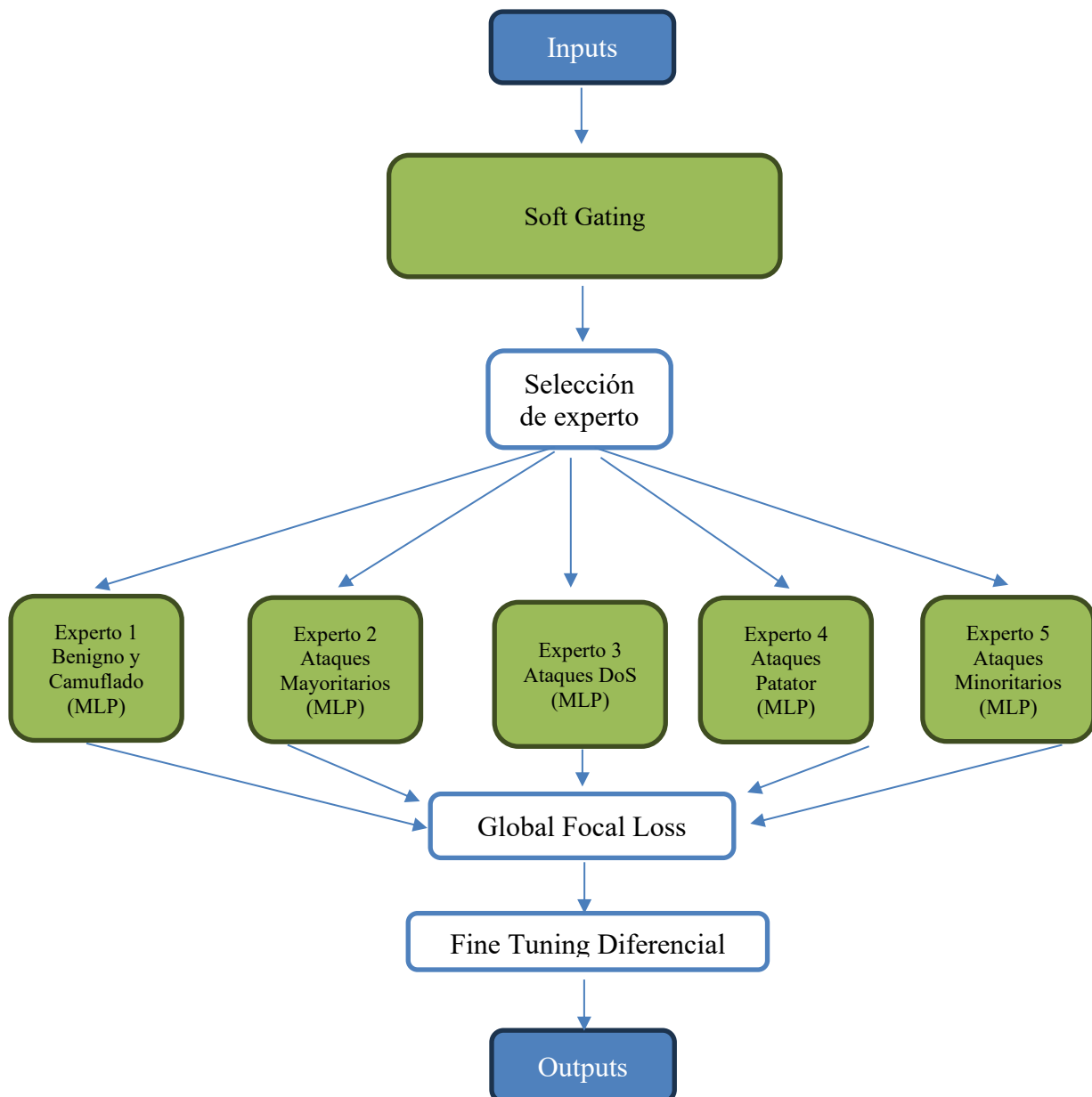


Figura 6.Arquitectura MoE “Soft Gating” con expertos Perceptrón Multicapa

3. Arquitectura de tipo “Soft Gating” con expertos tipo “TabTransformer”

- **Diseño y Preentrenamiento Individual de Expertos (“TabTransformer”):** La modificación estructural principal en esta tercera iteración radicó en la actualización de la arquitectura base de los expertos. Se reemplazó el Perceptrón Multicapa tradicional por la arquitectura “TabTransformer”. Este modelo integra mecanismos de autoatención que permiten capturar de manera más efectiva las relaciones complejas y las interacciones latentes dentro de las características tabulares del tráfico de red [32]. De manera análoga a las fases precedentes, los cinco expertos fueron pre-entrenados de forma independiente para especializarse en sus respectivos subconjuntos. Se conservó intacta la taxonomía de agrupación de ataques y los conjuntos de datos de cada experto.
- **Mecanismo de Enrutamiento:** Con base en la estabilidad y los resultados empíricos previos, se mantuvo el esquema de enrutamiento continuo (“Soft Gating”). La red de enrutamiento actúa como el coordinador central: procesa el vector de características entrante y asigna un coeficiente de confianza (peso probabilístico) a cada experto “TabTransformer”. La predicción final se calcula mediante la combinación de las salidas de todos los expertos, donde la influencia de cada subred está estrictamente ponderada por el nivel de confianza otorgado por el enrutador [31].
- **“Pipeline” de Entrenamiento y Refinamiento Diferencial:** Para el acoplamiento óptimo de este ensamble, se preservó la estrategia de ajuste concurrente utilizando tasas de aprendizaje diferenciales. Se habilitó la actualización simultánea de los gradientes tanto en la red de enrutamiento como en los expertos “TabTransformer”. Se asignó una tasa de aprendizaje superior al enrutador para acelerar la convergencia de su lógica de delegación, mientras que a los expertos se les aplicó una tasa significativamente reducida [28]. Esta configuración permitió la adaptación del modelo en conjunto,

evitando la degradación catastrófica y preservando las representaciones complejas de los ciberataques que los expertos adquirieron durante su fase de preentrenamiento.

Diagrama:

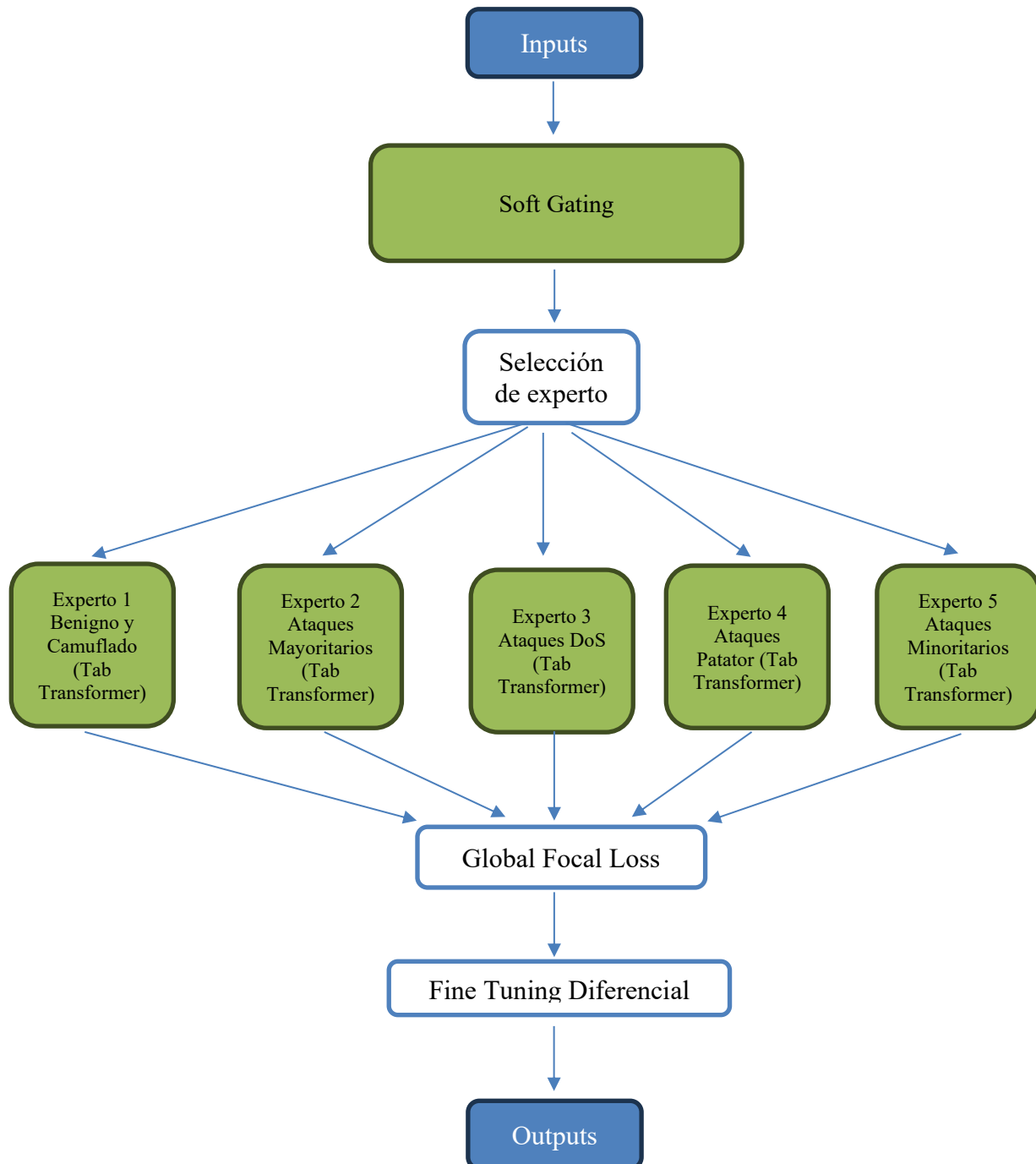


Figura 7. Arquitectura MoE "Soft Gating" con expertos "TabTransformer"

- **Tabla comparativa de Resultados de Arquitectura “Mixture of Experts” contra Resultados de Modelos Base**

En la tabla siguiente se presentan los resultados del modelo de referencia (sin técnicas de mejora) basado en “Random Forest”, junto con los resultados de los modelos base de “XGBoost” y de la red neuronal MLP, así como los obtenidos mediante el enfoque de “Mixture of Experts”, con el fin de establecer una comparativa directa entre los distintos enfoques evaluados:

Arquitectura	Sobremuestreo en sets de entrenamiento	F1 Macro	Recall Macro	Precisión Macro	Observación Principal
Random Forest	N/A	0.85	0.85	0.86	Mejor Arquitectura Base para Modelo tipo Random Forest
XGBoost	N/A	0.83	0.82	0.86	Mejor Arquitectura Base para Modelo tipo XGBoost
Red Neuronal MLP	N/A	0.77	0.75	0.92	Mejor Arquitectura Base para Modelo tipo Red Neuronal MLP
Mixture of Experts con Expertos MLP y Hard Gating	SMOTE	0.75	0.72	0.92	Alta Precisión, pero baja clasificación de ataques Web y Bot
	SMOTE-ENN	0.76	0.75	0.87	Alta Precisión, pero baja clasificación de ataques Web

	SMOTE Tomek	0.76	0.73	0.92	Performance similar al modelo con sets de tipo SMOTE
Mixture of Experts con Expertos MLP y Soft Gating	SMOTE	0.85	0.83	0.93	Modelo iguala rendimiento base, pero tiene dificultad en Web
	SMOTE-ENN	0.86	0.83	0.97	Primer modelo MoE que logra superar al modelo base en F1 Macro
	SMOTE Tomek	0.85	0.82	0.96	Modelo reduce mínimamente las métricas del MoE
Mixture of Experts con Expertos TabTransformer y Soft Gating	SMOTE	0.88	0.87	0.93	TabTransformer potencia la clasificación de clases difíciles
	SMOTE-ENN	0.89	0.87	0.96	El presente modelo logra las mejores métricas base para CIDS
	SMOTE Tomek	0.89	0.87	0.94	Modelo de rendimiento similar y superior al modelo base

Tabla 5. Métricas de Mejores Modelos MoE por arquitectura y sobremuestreo en subconjuntos de entrenamiento de expertos

De igual forma, complementado los resultados indicados en la tabla anterior, se procede a indicar del lado de Anexos las matrices de confusión (Ver Anexo H) para cada uno de los mejores resultados obtenidos en validaciones de set de pruebas a nivel de cada una de las tres arquitecturas MoE.

Desarrollo: Cuarta Parte: Análisis de Resultados

El análisis comparativo del rendimiento entre el mejor modelo de línea base y las arquitecturas “Mixture of Experts” (MoE) revela hallazgos importantes sobre el comportamiento de los clasificadores frente a un conjunto de datos con desbalance extremo, el cual en el presente caso para clasificación de intrusiones presenta un ratio de desbalance de 206645.18 y una entropía de 0.2837 [1], [2]. Cabe destacar que el presente escenario comparativo contempla estrictamente la comparación de modelos puros sin técnicas de mejora como pueden ser selección de características, ponderación de clases por pesos, entre otras. Bajo dicho contexto de comparación estricta de modelos puros la evaluación se centró primordialmente en la métrica “F1-Score” Macro para garantizar una medición equitativa que penalice los sesgos hacia el tráfico benigno [3].

Rendimiento de Modelos Base

La fase de experimentación inicial evidenció que los algoritmos fundamentados en ensambles de árboles poseen una robustez inherente frente al desbalance estructural de los datos [21]. El clasificador “Random Forest” se consolidó como la línea base más sólida, logrando un “F1-Score” Macro de 0.85 sin la aplicación de técnicas de preprocesamiento adicionales [19]. De manera similar, XGBoost, al basarse en estimadores de tipo árbol de decisión, alcanzó un rendimiento destacado con un F1-Score Macro de 0.83 en su configuración estándar [20].

En contraste, la Red Neuronal (Perceptrón Multicapa) mostró una severa limitación de convergencia bajo las mismas condiciones, estancándose en un F1-Score Macro de 0.77. Esta diferencia de 0.07 en la métrica confirma la dificultad técnica que presentan los modelos

tradicionales de aprendizaje profundo para procesar características tabulares heterogéneas prescindiendo de arquitecturas especializadas [27].

Posteriormente, se implementaron diversas técnicas de aprendizaje automático con el objetivo de optimizar el rendimiento predictivo de los modelos base. Los resultados empíricos revelaron los siguientes comportamientos:

- La inyección de muestras para las clases minoritarias no generó un impacto positivo significativo y, en ciertos escenarios, provocó una degradación en las métricas [2], [17]. Este fenómeno se atribuye a que la introducción de ruido sintético altera negativamente las fronteras de decisión de los vectores de ataque [6].
- Se incorporaron funciones de penalización para incrementar el peso de los errores en las clases minoritarias. La función de tipo “Balanced” produjo un incremento de 0.03 en el F1-Score Macro de los modelos basados en árboles, impactando positivamente en la capacidad de generalización [2]. La ineficacia de funciones de menor severidad sugiere que los escenarios de desbalance extremo exigen esquemas de penalización rigurosos [25].
- La aplicación de técnicas de selección de características no produjo mejoras significativas sobre las métricas establecidas [24], indicando que la dimensionalidad original preserva firmas de red necesarias para la detección [4].
- Ante la alta tasa de confusión en la clasificación de ataques Web, se procedió a unificar estas categorías en una sola etiqueta. Esta reestructuración taxonómica elevó sustancialmente el rendimiento, alcanzando un F1-Score Macro de hasta 0.93. Dicho incremento se justifica técnicamente debido a que estas intrusiones comparten

arquitecturas y firmas de tráfico a nivel de red análogas, lo que dificulta la distinción granular exacta por parte de un modelo general [26].

Rendimiento de “Mixture of Experts”:

El desarrollo de una arquitectura de inteligencia artificial basada en el paradigma de “Mixture of Experts” respondió a la necesidad de mitigar las deficiencias de convergencia de los modelos de aprendizaje profundo convencionales frente a datos tabulares con desbalance extremo [17], [28]. Mientras que una red neuronal estándar se estancó en un “F1-Score Macro” de 0.77, el enfoque delegativo de MoE permitió escalar la capacidad predictiva mediante la especialización de subredes [7].

La progresión del diseño arquitectónico, dividida en tres fases fundamentales, ilustra el proceso de optimización técnica necesario para superar la robustez de los ensambles de árboles de decisión.

- En la primera fase de experimentación, se implementó una arquitectura de enrutamiento estricto o “Hard Gating” Top-K, empleando seis expertos configurados como Perceptrones Multicapa (MLP) [30]. Bajo este esquema, la red de enrutamiento emitía una máscara para seleccionar exclusivamente a los expertos con mayor probabilidad condicional, aislando al resto del ensamble. Los resultados de esta iteración arrojaron un “F1-Score” Macro máximo de 0.76, utilizando técnicas de sobremuestreo híbrido como SMOTE-Tomek y SMOTE-ENN [16]. Desde una perspectiva analítica, este modelo no logró alcanzar la línea base del “Random Forest” debido a las limitaciones intrínsecas del enrutamiento rígido. Al forzar una decisión excluyente en la compuerta,

se descarta información probabilística crítica sobre ataques que comparten firmas de red solapadas, lo que introduce ruido predictivo y dificulta la resolución de ambigüedades en muestras de ataque complejos.

- La segunda iteración marcó un punto de inflexión técnica al transicionar hacia un mecanismo de “Soft Gating” con expertos MLP y un refinamiento basado en tasas de aprendizaje diferenciales [31]. En esta configuración, la compuerta operó mediante una capa “Softmax” continua, permitiendo que la predicción final fuera el resultado de una colaboración ponderada entre todos los expertos. Además, se optimizó la taxonomía del ensamble reduciendo la partición a cinco subconjuntos especializados, agrupando clases de ataques con patrones similares para potenciar la capacidad de especialización [1]. Esta arquitectura alcanzó un “F1-Score” Macro de 0.86, logrando por primera vez superar el objetivo de sobrepasar la línea base del modelo “Random Forest”. La mejora se justifica técnicamente por el enrutamiento probabilístico, que permite mitigar errores individuales mediante el consenso del ensamble, y por el ajuste fino diferencial que aceleró la capacidad de delegación de la compuerta sin comprometer los pesos pre-entrenados de los expertos.
- En la tercera fase de refinamiento, se mantuvo el esquema de enrutamiento suave, pero se ejecutó una actualización estructural profunda al reemplazar los perceptrones tradicionales por bloques “TabTransformer” en cada experto [27]. Esta modificación integró mecanismos de autoatención, diseñados para capturar de forma más efectiva las relaciones latentes y las interacciones multivariadas complejas dentro de las características tabulares del tráfico de red. Los resultados obtenidos consolidaron la superioridad de la arquitectura MoE sobre la línea base comparativa, obteniendo un “F1-Score” Macro de 0.89 e incrementando drásticamente la Precisión Macro hasta un 0.96 con el uso de SMOTE-Tomek [16]. La ganancia sustancial en las métricas

evidencia que los mecanismos de atención fueron determinantes para optimizar la reducción de falsos positivos, validando al ensamble MoE con “TabTransformers” como la configuración más robusta para la detección de intrusiones en entornos de desbalance crítico [8], [29].

Conclusiones

- Se considera que el desarrollo posee una alta pertinencia profesional debido a que la digitalización acelerada exige sistemas de defensa capaces de discriminar tráfico malicioso en infraestructuras computacionales críticas donde la proporción de intrusiones es estadísticamente mínima frente al tráfico benigno lo cual es requerido en sistemas a nivel nacional y global.
- Se concluyó que las estrategias de depuración de datos deben preservar el desbalance estructural original (eliminando duplicados y nulos exclusivamente de la clase mayoritaria) para evitar la introducción de sesgos artificiales que perjudiquen el entrenamiento en escenarios realistas para el caso de un “Dataset” con desbalance extremo, como es el presente caso.
- Se reconoció que los modelos basados en árboles clasificadores permiten obtener un mejor reconocimiento de patrones de características a la hora de generar una clasificación adecuada de las intrusiones obteniendo métricas más altas en sus modelos base y mejorados con técnicas en comparación al uso de redes neuronales estándar para objetivos de identificación de intrusiones esto siendo que los modelos base de “Random Forest” y “XGBoost” generaron F1-scores macro de 0.85 y 0.83 mientras que el modelo

de tipo red neuronal estándar de perceptrón multicapa solo llegó a un F1-score macro de 0.77

- Se validó que, ante escenarios de desbalance extremo, la asignación de penalizaciones mediante funciones de costo resulta ser una técnica de regularización más estable y efectiva que la inyección de ruido sintético puro, la cual tiende a degradar las fronteras de decisión.
- Se determinó que la agrupación de intrusiones intra-dominio, como las diferentes variantes de ataques Web (“Brute Force”, “SQL Injection”, “Cross Site Scripting”), reduce sustancialmente la confusión del clasificador y eleva el “F1-Score” Macro hasta 0.93 en modelos base, debido al alto nivel de similitud en sus características de red.
- Se evidenció que las técnicas de reducción de dimensionalidad y selección de características no aportan mejoras predictivas, concluyendo que la dimensionalidad original contiene firmas sutiles e indispensables para la detección de ataques minoritarios.
- Se implementó exitosamente una arquitectura de Inteligencia Artificial basada en “Mixture of Experts” (MoE) que superó las limitaciones de convergencia de las redes neuronales estándar y logró sobrepasar el rendimiento predictivo de clasificadores robustos como “Random Forest” en el tratamiento de datos tabulares.
- Se demostró empíricamente que la integración de bloques “TabTransformer” bajo un esquema de enrutamiento continuo incrementa la capacidad del modelo para capturar relaciones multivariadas complejas, logrando estabilizar el “F1-Score” Macro en 0.89 y maximizar la Precisión Macro hasta 0.96 lo cual mejora de manera significativa a las métricas obtenidas en el modelo base de tipo “Random Forest”.
- Se estableció que el uso de tasas de aprendizaje independientes y concurrentes para la red de enrutamiento y los expertos durante la fase de ajuste fino global es fundamental

para lograr la sincronización del ensamble, evitando la degradación de las representaciones previamente adquiridas.

- Se validó que el mecanismo de “Soft Gating” es técnicamente superior al esquema de “Hard Gating” para la clasificación de intrusiones, ya que permite una colaboración ponderada entre expertos que mitiga el ruido predictivo en muestras con firmas de red solapadas. Mientras que el enrutamiento estricto descarta información probabilística crítica, el esquema basado en “Softmax” garantiza un consenso que eleva la capacidad de generalización del ensamble frente a ataques sofisticados
- Se determinó que la efectividad de la arquitectura MoE reside en una segmentación taxonómica de subconjuntos basada en la dificultad de distinción de las etiquetas y en la integración de modelos de expertos adaptados a la estructura del tráfico. En este contexto, la transición de perceptrones multicapa hacia bloques “TabTransformer” permitió explotar mecanismos de autoatención para capturar relaciones complejas entre clases, optimizando el “F1-score” macro en la detección de anomalías dentro de datos tabulares de red

Recomendaciones

Con base en la metodología ejecutada y la validación empírica de los resultados en la clasificación de intrusiones cibernéticas, se establecen las siguientes recomendaciones de buenas prácticas para la replicabilidad y optimización de futuros proyectos e investigaciones de esta índole:

- Se recomienda mantener la distribución y el desbalance original del conjunto de datos. La eliminación de registros duplicados y la exclusión de nulos estructurales deben aplicarse exclusivamente sobre la clase mayoritaria (tráfico benigno), con el fin de evitar la alteración o pérdida de información representativa de los ataques minoritarios

- Es imperativo aplicar estrategias de imputación condicionadas a cada clase dado que en ciberseguridad los datos faltantes no son aleatorios, imputar variables de forma generalizada alteraría y difuminaría el comportamiento específico de cada tipo de ataque.
- Se recomienda evitar técnicas de reducción de dimensionalidad que combinen variables, como el Análisis de Componentes Principales (PCA), ya que los resultados experimentales evidencian que la disminución de características degrada las métricas de clasificación. Este efecto, observado en la serie de experimentos 4, indica que conservar la mayoría o la totalidad de las características durante el entrenamiento mejora la discriminación de clases minoritarias al mantener un espacio de representación más completo [2].
- Para el manejo del desbalance extremo en el “Dataset” de intrusiones, resulta una práctica más estable priorizar el uso de penalizaciones dinámicas en lugar del sobremuestreo sintético. La introducción excesiva de ruido sintético tiende a degradar las fronteras de decisión en modelos basados en árboles y redes neuronales
- Ante la alta dimensionalidad y el elevado costo computacional de los experimentos, se debe utilizar optimización bayesiana como Optuna en lugar de métodos de cuadrícula exhaustiva como Grid Search, permitiendo priorizar generación de modelos más prometedores y podar configuraciones ineficientes
- En el diseño de la compuerta integradora, se recomienda evitar el enrutamiento rígido (“Hard Gating”). En su lugar, el uso de capas “Softmax” para emitir probabilidades condicionales permite que la predicción final se beneficie de la ponderación simultánea de todos los expertos, lo cual es crítico frente a ataques con firmas solapadas
- La partición de los subconjuntos de datos para entrenar a cada experto no debe ser aleatoria ni estrictamente por volumen. Es una buena práctica analizar los errores

(matrices de confusión) de los modelos de línea base para agrupar deliberadamente clases que el modelo confunde entre sí (como tráfico benigno con ataques camuflados), forzando al experto a especializarse en distinguir esos patrones.

- Para procesar datos de tráfico de red, se recomienda prescindir de redes MLP tradicionales y dotar a los expertos de arquitecturas especializadas en datos tabulares, como “TabTransformer”. Esto asegura que cada subred pueda capturar interacciones latentes complejas, logrando optimizar las métricas de clasificación.

Trabajo Futuro

Con base en lo desarrollado en el presente proyecto, donde se llevaron a cabo implementaciones de modelos base y arquitecturas de tipo “Mixture of Experts” (MoE) especializadas en la clasificación de distintos tipos de intrusiones, y respecto al alcance obtenido, se plantean los siguientes puntos como potencial trabajo futuro para continuar la línea de mejora continua de esta investigación:

- Implementación de arquitecturas de modelos base y de tipo “Mixture of Experts” sobre nuevos “Dataset” de intrusiones cibernéticas para comprobar si el presente trabajo se puede extender a muestras recabadas y analizadas en otras investigaciones. A su vez, se plantea extender el “Dataset” CIC-IDS2017 (de la Universidad de New Brunswick) empleado en este estudio, fusionándolo con otros conjuntos de datos mediante la homologación a nivel de características y “labels”. Todo esto con el objetivo de generalizar al máximo la arquitectura del modelo, con miras a su implementación en entornos reales para la detección de intrusiones en tiempo real.
- Generación de una API de consumo e implementación sobre un entorno o sitio web de “Pentesting” para validar, en situaciones de simulación de ataques, la capacidad de

respuesta del modelo de detección. La finalidad es utilizar el output generado para determinar y aplicar reglas dinámicas a nivel de capa de red y de aplicación, impidiendo el tráfico con características maliciosas. De esta forma, se busca validar la generalización y utilidad del modelo en un escenario operativo y realista.

- Validación de arquitecturas Mixture of Experts incorporando modelos de red neuronal y arquitecturas mixtas diferentes a nivel de los expertos. En el presente proyecto se emplearon arquitecturas basadas en Perceptrón Multicapa (MLP) y “TabTransformer”; no obstante, es importante evaluar el rendimiento de enfoques como 1D-CNN o “FT-Transformer”. Esto permitirá valorar si poseen una mejor capacidad para modelar relaciones no lineales complejas, lo que podría traducirse en una mejora significativa del rendimiento global del sistema.
- Evaluación de las arquitecturas “Mixture of Experts” con las distintas técnicas implementadas en la serie de experimentos de los modelos base a manera de validar el rendimiento de estos modelos ante la implementación de funciones de pesos, selección de características y agrupación de clases Web. Esto con el objetivo de verificar si los rendimientos obtenidos son mejorados o sufren degradación a la par que se complementaría las conclusiones obtenidas con base en las pruebas realizadas con dichas técnicas para los modelos base.
- En consideración de que a pesar de la complejidad de los modelos existió dificultad en potenciar las métricas de clasificación de los ataques de tipo Bot debido a que contienen firmas de características similares al tráfico benigno se considera que se debe ampliar la cantidad de muestras de intrusiones de tipo Bot en el “Dataset” de entrenamiento para mejorar la detección de este tipo de ataque. Para este propósito se puede tomar como referencia el “Dataset” de la investigación del IEEE “A review and analysis of the bot-iot “Dataset”” de Peterson, Leevy y Khoshgoftaar [32]

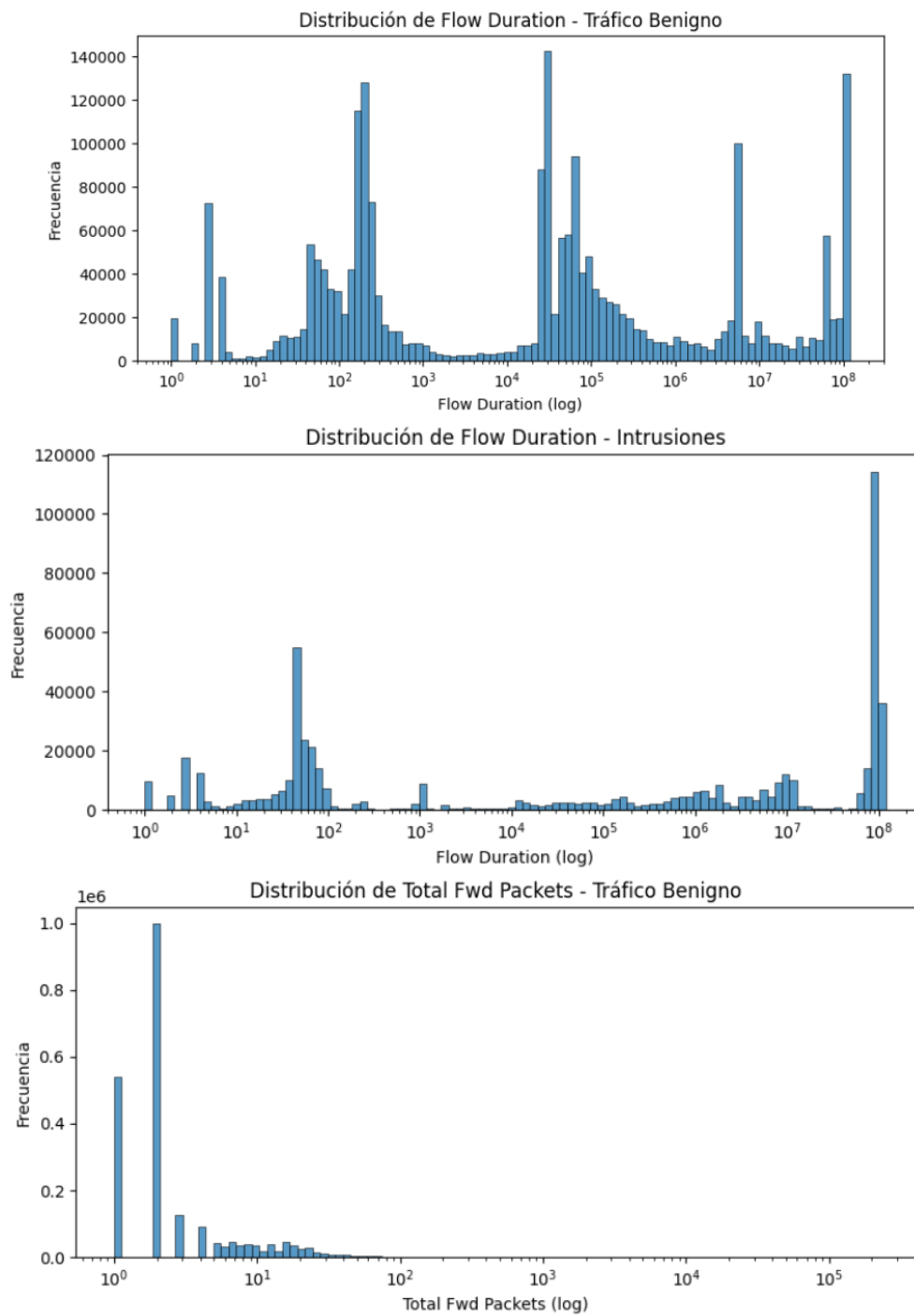
Referencias bibliográficas

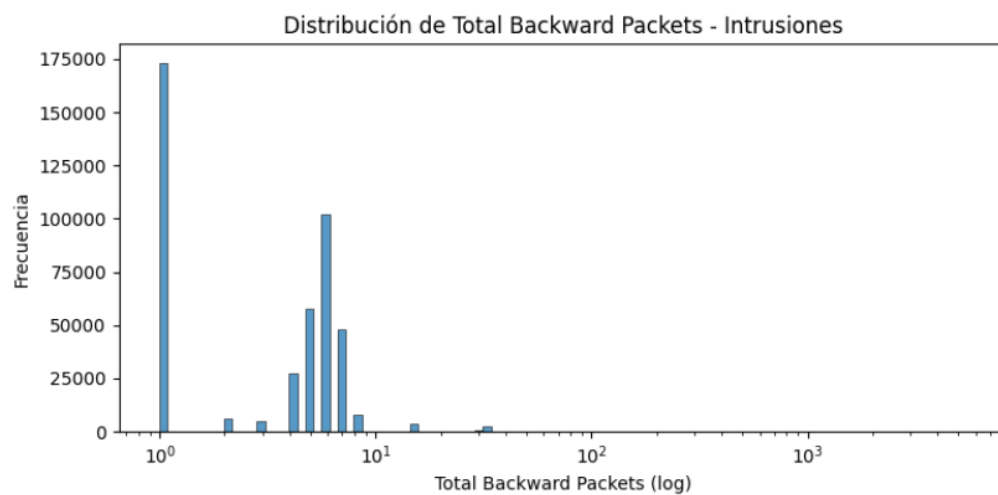
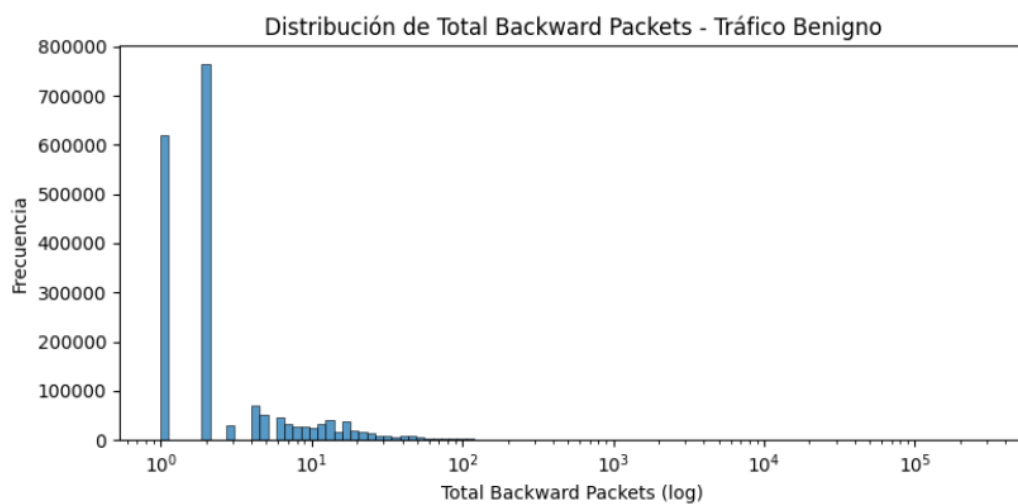
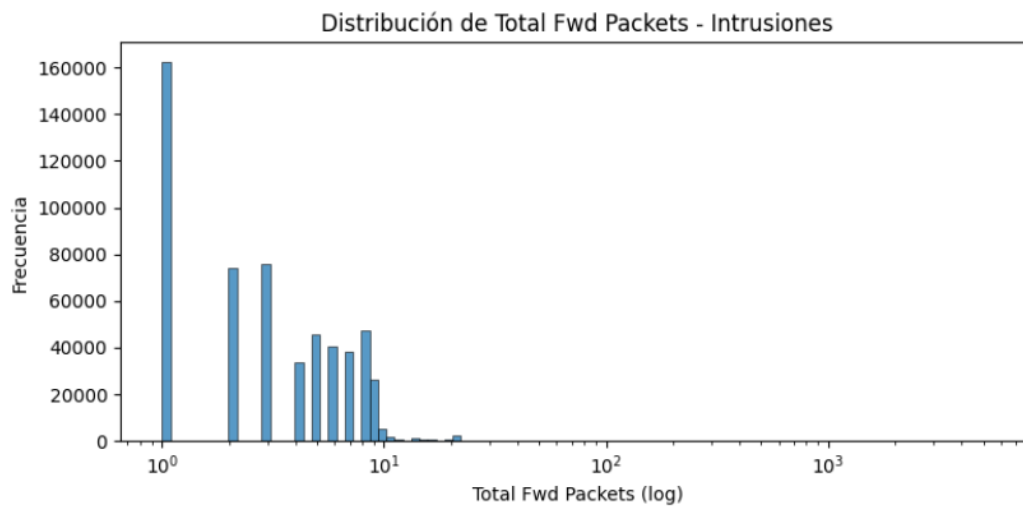
1. P. Mishra, V. Varadharajan, U. Tupakula and E. S. Pilli, "A Detailed Investigation and Analysis of Using Machine Learning Techniques for Intrusion Detection," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 686–728, Firstquarter 2019, doi: 10.1109/COMST.2018.2847722.
2. H. He and E. A. Garcia, "Learning from Imbalanced Data," in *Data Mining and Knowledge Discovery Handbook*, 2nd ed., O. Maimon and L. Rokach, Eds. Boston, MA, USA: Springer, 2020, pp. 875–896.
3. S. Raschka, *Model Evaluation, Model Selection, and Algorithm Selection in Python Machine Learning*, 3rd ed., Birmingham, UK: Packt Publishing, 2019.
4. A. Thakkar and R. Lohiya, "Fusion of statistical importance for feature selection in deep neural network-based intrusion detection system," *Information Fusion*, vol. 90, pp. 353–363, 2023, doi: 10.1016/j.inffus.2022.09.026.
5. U. K. Lilhore, S. Dalal, and S. Simaiya, "A cognitive security framework for detecting intrusions in IoT and 5G utilizing deep learning," *Computers & Security*, vol. 136, p. 103560, 2024, doi: 10.1016/j.cose.2023.103560.
6. H. Zhang, L. Huang, C. Q. Wu, and Z. Li, "An effective convolutional neural network based on SMOTE and Gaussian mixture model for intrusion detection in imbalanced Dataset," *Computer Networks*, vol. 177, p. 107315, 2020, doi: 10.1016/j.comnet.2020.107315.
7. Y. Han et al., "Dynamic neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2022, doi: 10.1109/TPAMI.2021.3117837.
8. L. Ilias, G. Doukas, V. Lamprou, C. Ntanos, and D. Askounis, "Convolutional neural networks and mixture of experts for intrusion detection in 5G networks and beyond," *Frontiers in Artificial Intelligence*, vol. 8, Art. no. 1708953, 2026, doi: 10.3389/frai.2025.1708953.
9. Canadian Institute for Cybersecurity, University of New Brunswick, CIC-IDS2017 Intrusion Detection Evaluation Dataset, Disponible en: <https://www.unb.ca/cic/Datasets/ids-2017.html>
10. I. Sharafaldin et al., *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
11. S. G. Liao, Y. Lin, D. D. Kang, D. Chandra, J. Bon, N. Kaminski, et al., "Missing value imputation in high-dimensional phenomic data: imputable or not, and how?," *BMC Bioinformatics*, vol. 15, no. 1, p. 346, 2014.
12. F. Hinder et al., *ACM Comput. Surv.*, vol. 56, no. 4, pp. 1–35, 2023.
13. M. Ring et al., *Comput. Secur.*, vol. 86, pp. 147–167, 2019.
14. S. García et al., *Data Preprocessing in Data Mining*, Springer, 2015.
15. Wu, K., Qu, H., and Huang, C., "A Network Intrusion Detection Method Incorporating Bayesian Attack Graph and Incremental Learning Part," *Future Internet*, vol. 15, no. 4, p. 128, 2023. doi: 10.3390/fi15040128.
16. D. Kasina, V. S. Reddy, S. Rajamanickam, J. Jose, and B. Palani, "A novel intrusion detection system for class imbalance Datasets using hybrid sampling with deep learning techniques," *Information Sciences*, vol. 741, p. 123208, Jun. 2026. doi: 10.1016/j.ins.2026.123208.
17. J. M. Johnson and T. M. Khoshgoftaar, "Survey on deep learning with class imbalance," *J. Big Data*, vol. 6, no. 1, pp. 1–54, Dec. 2019. doi: 10.1186/s40537-019-0192-5.

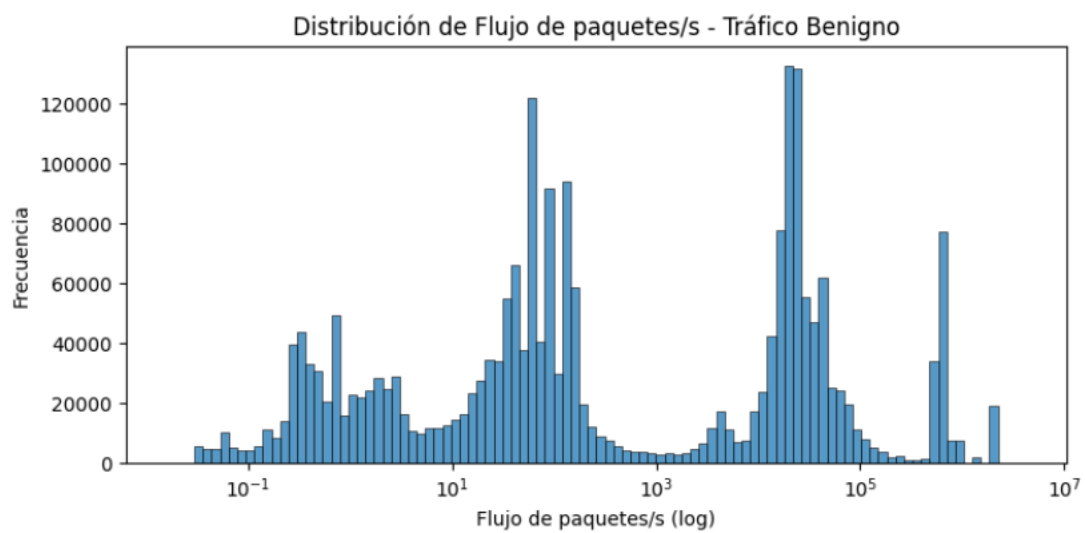
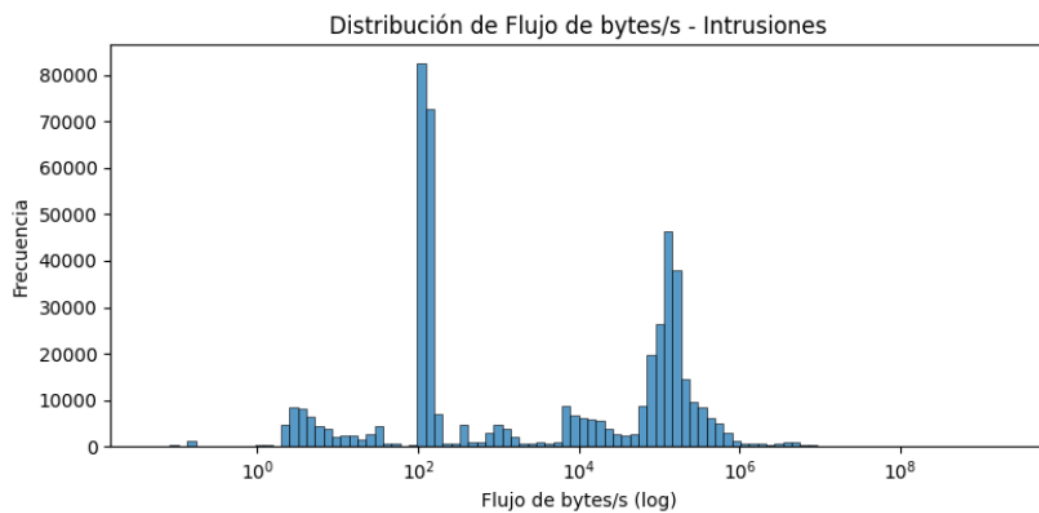
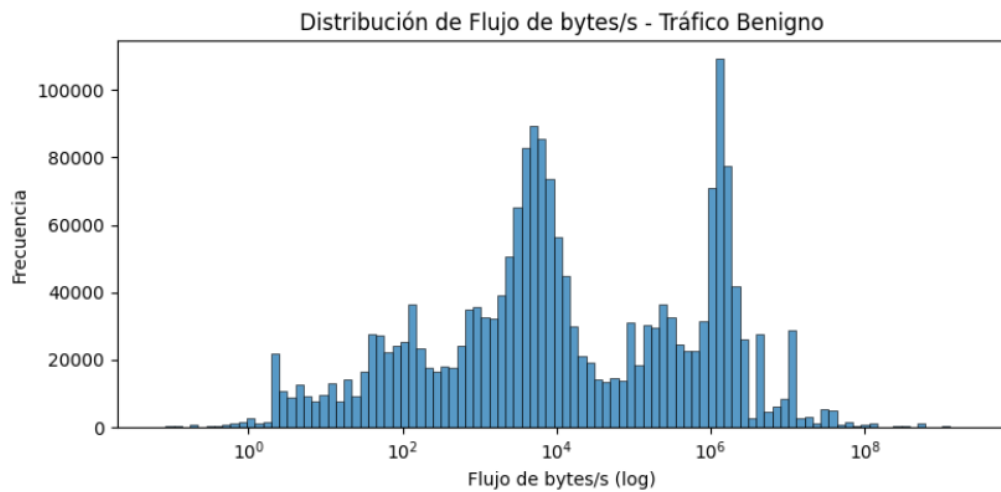
18. T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., 2019, pp. 2623–2631.
19. P. V. Chavan and N. V. Alone, "Optimizing Intrusion Detection with Random Forest: A High-Accuracy Approach using CIC-IDS2017" *International Journal of Computer Applications*, vol. 187, no. 3, pp. 17–22, May 2025.
20. T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD), San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
21. T. H. Sow and M. Adda, "Enhancing IDS performance through a comparative analysis of Random Forest, XGBoost, and Deep Neural Networks," *Machine Learning with Applications*, vol. 22, Art. no. 100738, 2025, doi: 10.1016/j.mlwa.2025.100738.
22. R. Saini, G. Mussbacher, J. L. Guo y J. Kienzle, "Machine learning-based incremental learning in interactive domain modelling," en *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems*, Oct. 2022, pp. 176–186.
23. A. Furlong, X. Zhao y R. Salko Jr., "Comparison of hybrid and pure machine learning models in limited data scenarios," Oak Ridge National Laboratory (ORNL), Oak Ridge, TN, USA, 2024
24. M. J. Siraj, T. Ahmad y R. M. Ijtihadie, "Analyzing ANOVA F-test and Sequential Feature Selection for Intrusion Detection Systems," *International Journal of Advances in Soft Computing & Its Applications*, vol. 14, no. 2, 2022.
25. M. Mulyanto, M. Faisal, S. W. Prakosa y J. S. Leu, "Effectiveness of focal loss for minority classification in network intrusion detection systems," *Symmetry*, vol. 13, no. 1, p. 4, 2020.
26. R. Singh, H. Kumar, R. K. Singla y R. R. Ketti, "Internet attacks and intrusion detection system: A review of the literature," *Online Information Review*, vol. 41, no. 2, pp. 171–184, 2017.
27. D. McElfresh, S. Khandagale, J. Valverde, V. Prasad C, G. Ramakrishnan, M. Goldblum y C. White, "When do neural nets outperform boosted trees on tabular data?," *Advances in Neural Information Processing Systems*, vol. 36, pp. 76336–76369, 2023.
28. J. Liu, P. Tang, W. Wang, Y. Ren, X. Hou, P. A. Heng, et al., "A survey on inference optimization techniques for mixture of experts models," *ACM Computing Surveys*, vol. 58, no. 10, pp. 1–37, 2026.
29. J. Wang, L. Nie, J. Duan, H. Zhao y C. P. Chen, "MFBLS: A Mixture-of-Experts-Based Fuzzy Broad Learning System for Tackling Imbalanced Datasets," *IEEE Transactions on Fuzzy Systems*, 2025.
30. Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., ... & Laudon, J. (2022). Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35, 7103-7114.
31. H. Nguyen, T. Nguyen y N. Ho, "Demystifying softmax gating function in Gaussian mixture of experts," *Advances in Neural Information Processing Systems*, vol. 36, pp. 4624–4652, 2023.
32. J. M. Peterson, J. L. Leevy y T. M. Khoshgoftaar, "A review and analysis of the Bot-IoT Dataset," en Proc. IEEE Int. Conf. Service-Oriented System Engineering (SOSE), Aug. 2021, pp. 20–27, doi: 10.1109/SOSE52839.2021.00007

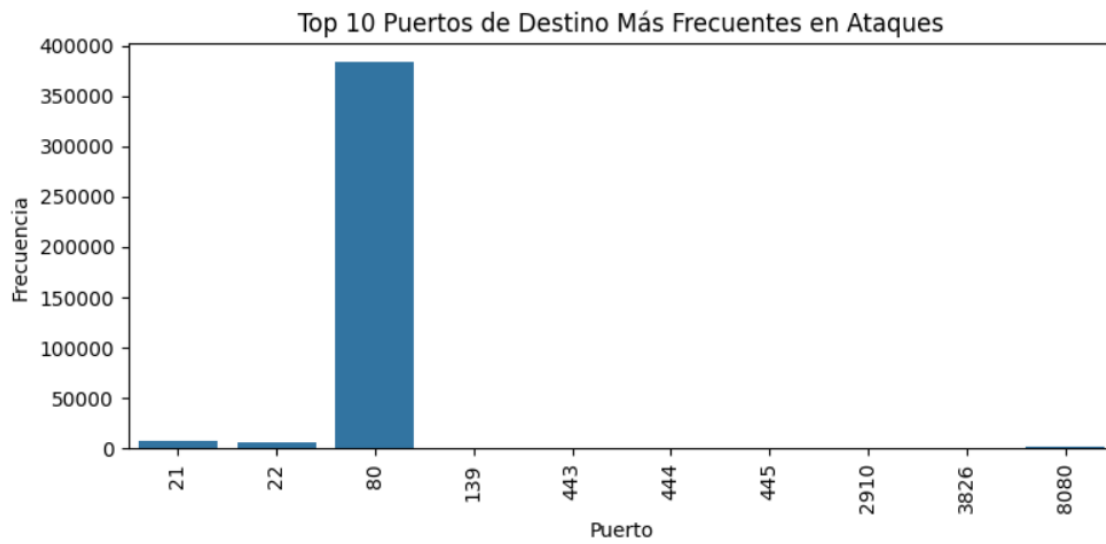
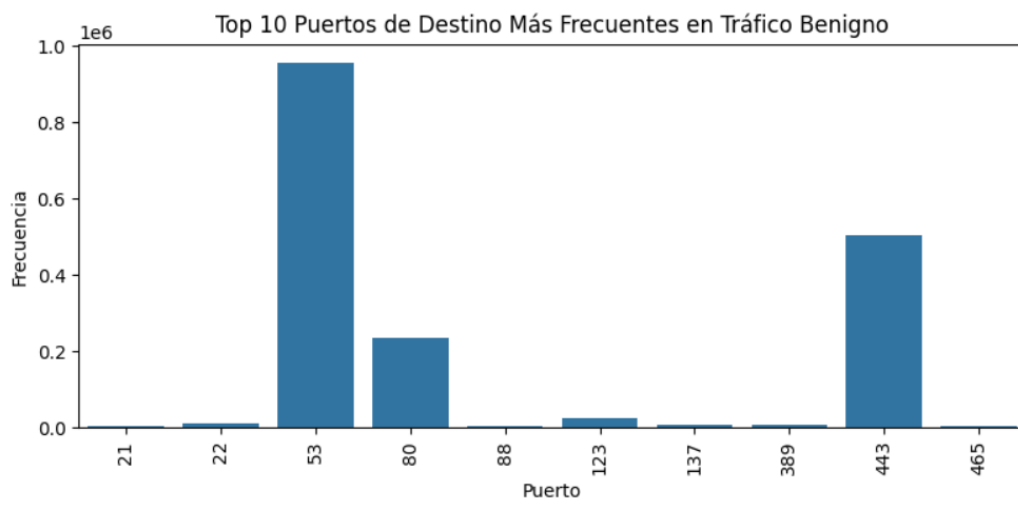
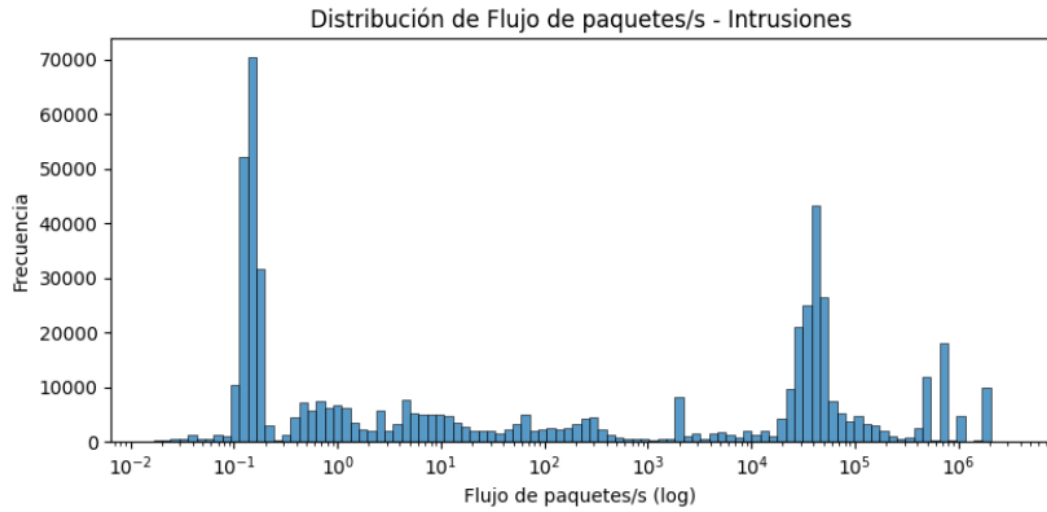
Anexo A: Conteo Inicial de Muestras por Clase

	count	percent
label		
BENIGN	2273097	80.300366
DoS Hulk	231073	8.162981
PortScan	158930	5.614427
DDoS	128027	4.522735
DoS GoldenEye	10293	0.363615
FTP-Patator	7938	0.280421
SSH-Patator	5897	0.208320
DoS slowloris	5796	0.204752
DoS Slowhttptest	5499	0.194260
Bot	1966	0.069452
Web Attack Brute Force	1507	0.053237
Web Attack XSS	652	0.023033
Infiltration	36	0.001272
Web Attack Sql Injection	21	0.000742
Heartbleed	11	0.000389

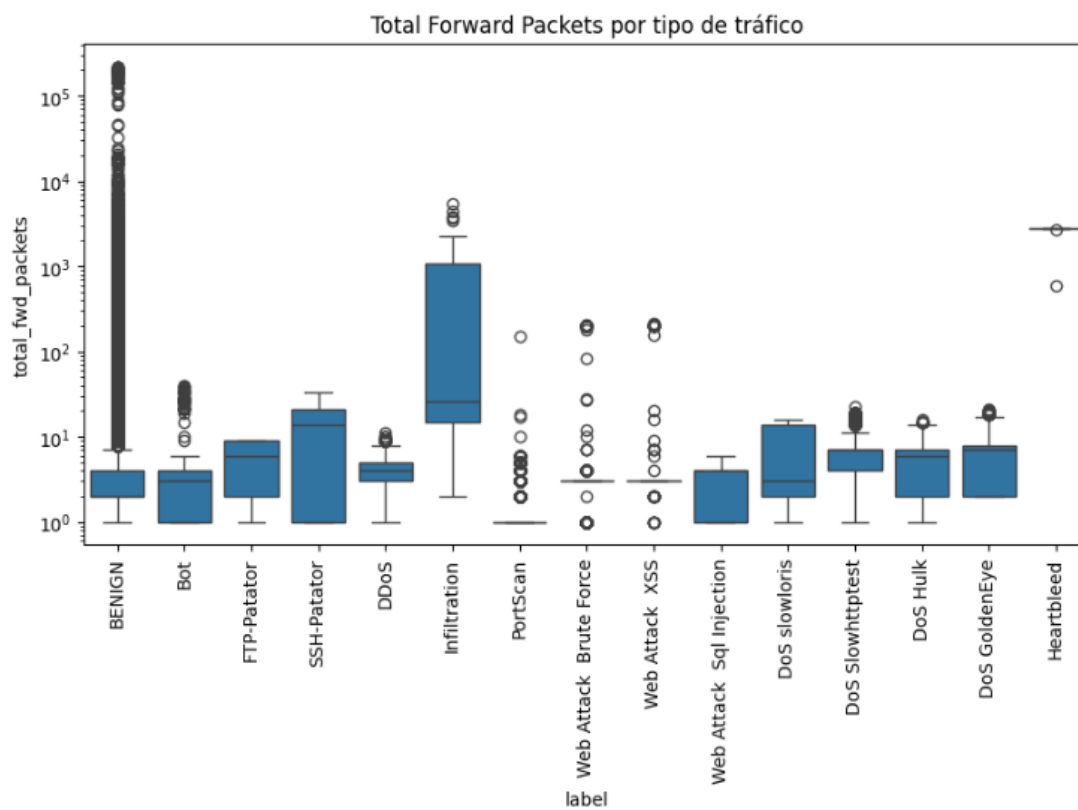
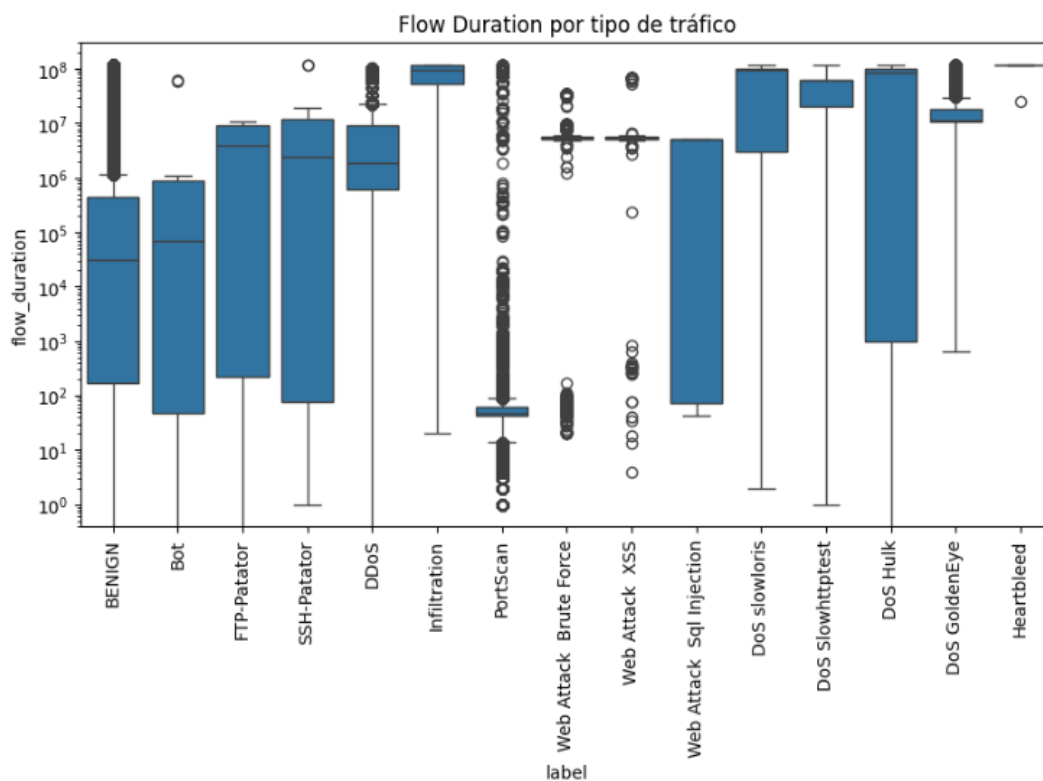
Anexo B: Gráficos de Distribución de Features para tráfico Benigno y Ataques

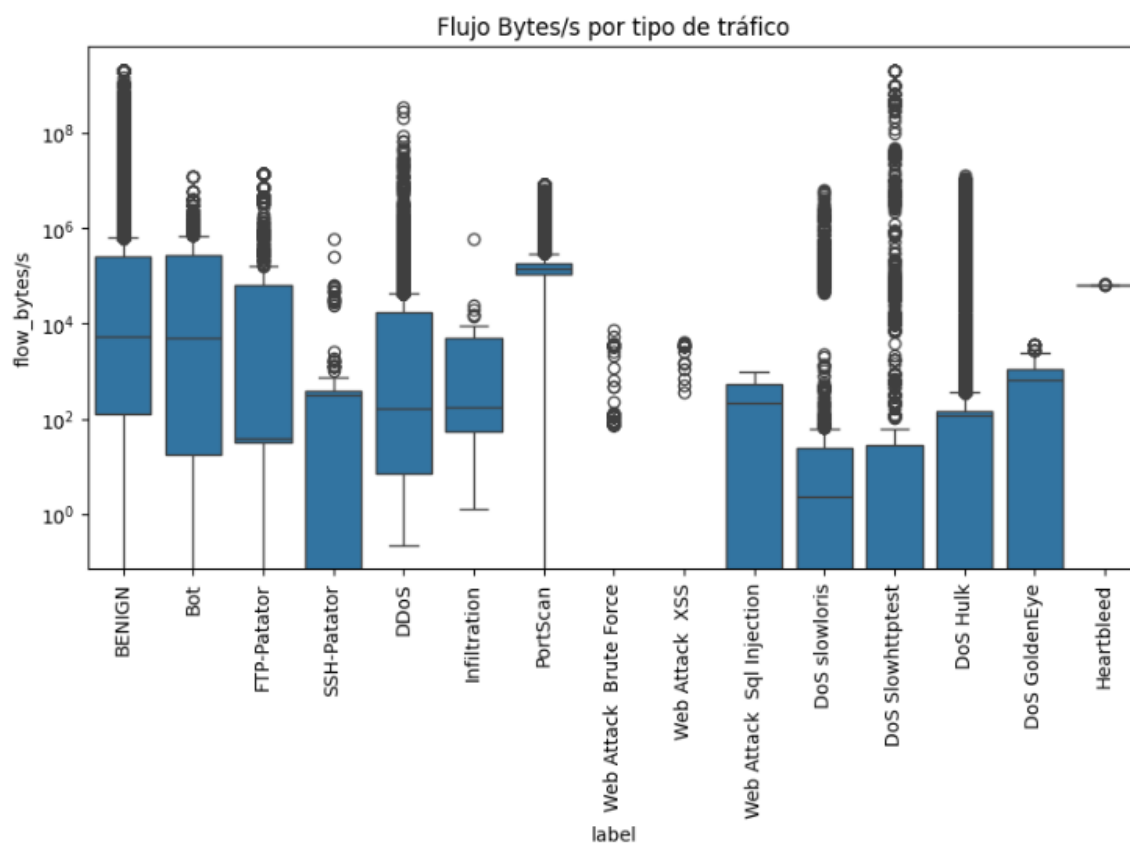
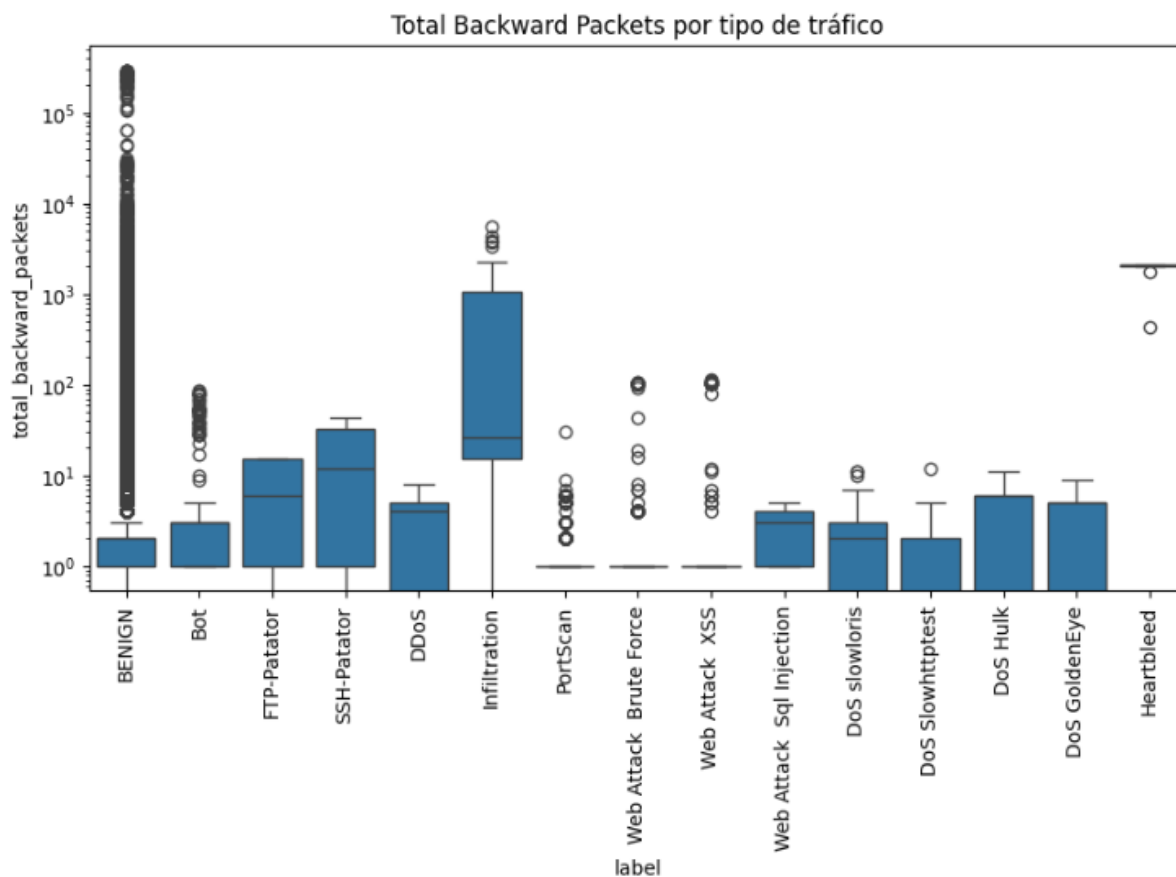


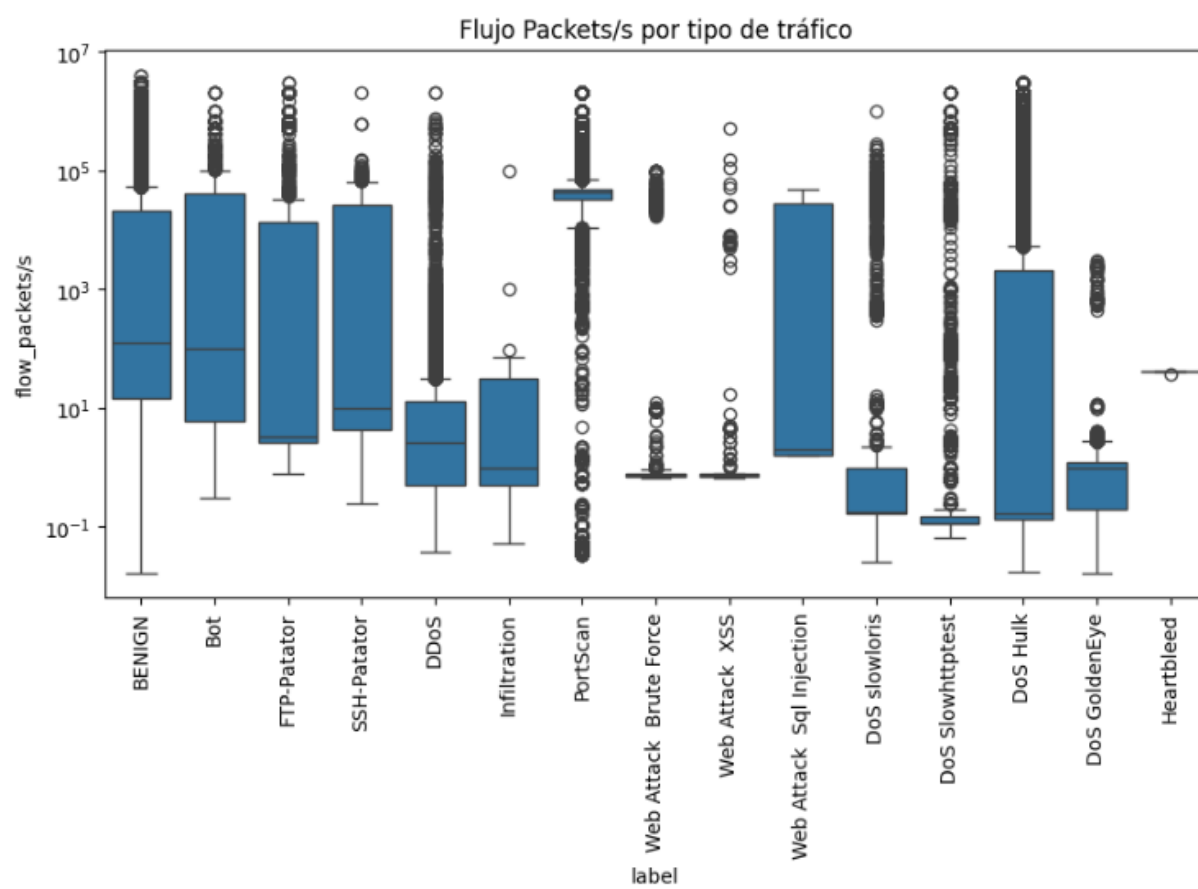




Anexo C: Boxplots de Features por Clase para Flujos Benignos e Intrusiones







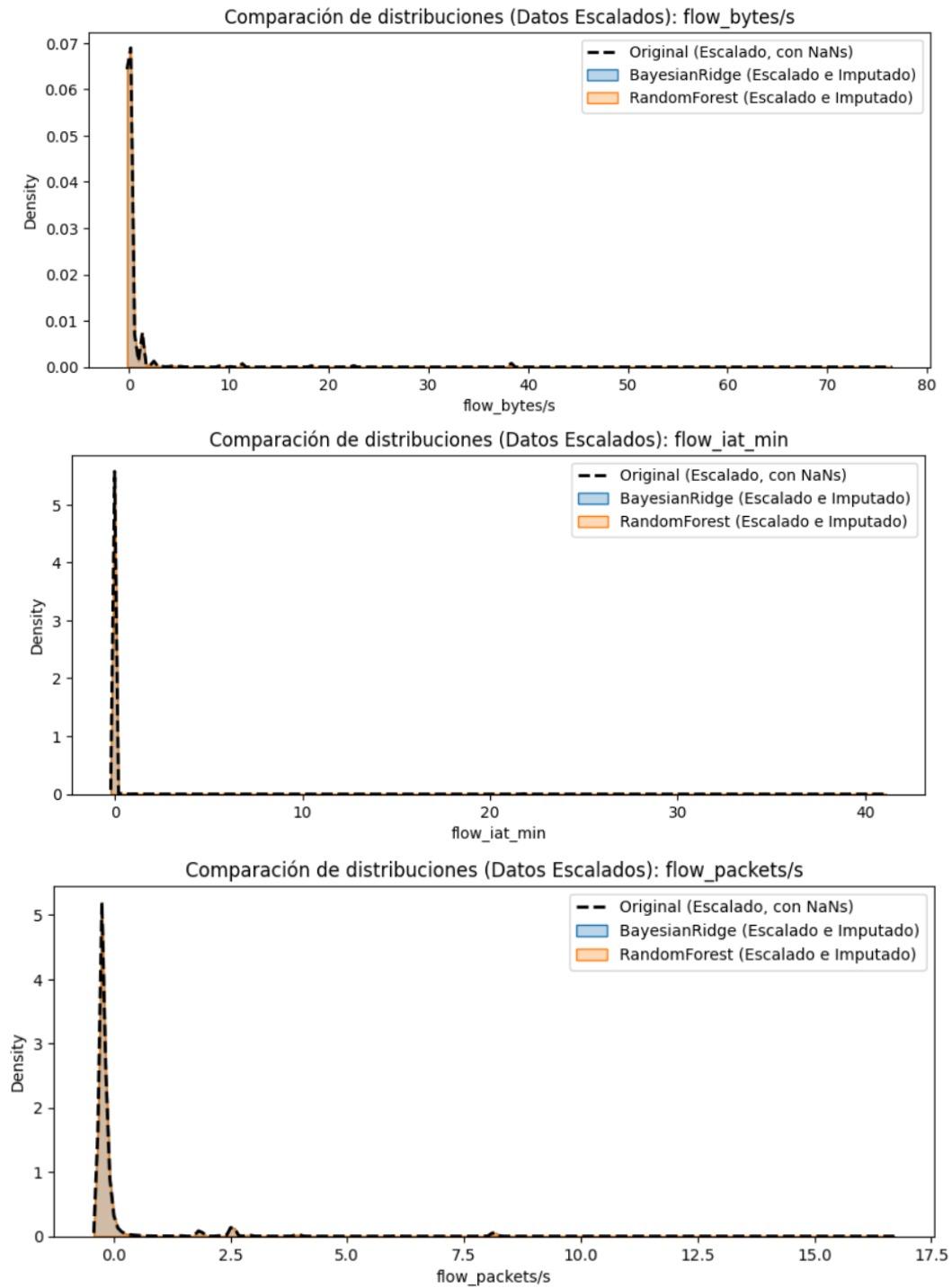
Anexo D: Tabla de Resumen Antes de la Limpieza de Datos

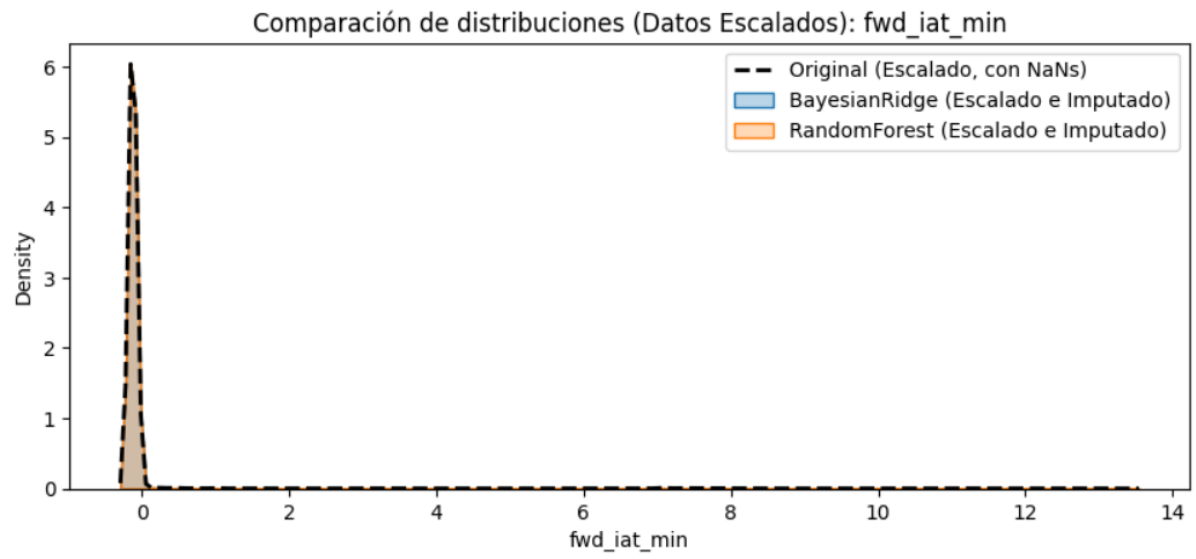
	tipo_datos	no_nulos	nulos	alores_unico	count	mean	std	min	25%	50%	75%	max
flow_bytes/s	float64	2829385	1358	1593908	2829385	inf		-261000000	119.319731	4595.54913	166666.667	inf
flow_duration	int64	2830743	0	1050899	2830743	14785663.9	33653744.1	-13	155	31316	3204828.5	1199999998
destination_port	int64	2830743	0	53805	2830743	8071.4825	18283.6322	0	53	80	443	65535
total_backward_packets	int64	2830743	0	1747	2830743	10.3937701	997.388269	0	1	2	4	291922
total_length_of_fwd_packets	int64	2830743	0	17928	2830743	549.302415	9993.58923	0	12	62	187	12900000
total_length_of_bwd_packets	int64	2830743	0	64698	2830743	16162.6425	2263088.05	0	0	123	482	655453030
total_fwd_packets	int64	2830743	0	1432	2830743	9.36115995	749.672792	1	2	2	5	219759
fwd_packet_length_max	int64	2830743	0	5279	2830743	207.599941	717.184842	0	6	37	81	24820
fwd_packet_length_min	int64	2830743	0	384	2830743	18.7136561	60.3393454	0	0	2	36	2325
fwd_packet_length_std	float64	2830743	0	253909	2830743	68.9101335	281.187092	0	0	0	26.1629509	7125.59685
fwd_packet_length_mean	float64	2830743	0	99716	2830743	58.2019385	186.091157	0	6	34	50	5940.85714
bwd_packet_length_max	int64	2830743	0	4838	2830743	870.849455	1946.36738	0	0	79	280	19530
bwd_packet_length_min	int64	2830743	0	583	2830743	41.0495792	68.8626026	0	0	0	77	2896
bwd_packet_length_mean	float64	2830743	0	147614	2830743	305.949281	605.2568	0	0	72	181	5800.5
bwd_packet_length_std	float64	2830743	0	248869	2830743	335.325721	839.693218	0	0	0	77.9405368	8194.66049
flow_packets/s	float64	2830743	0	1240164	2830743	inf		-2000000	3.44622571	110.668437	23255.814	inf
flow_iat_mean	float64	2830743	0	1166311	2830743	1298448.71	4507944.17	-13	63.6666667	11438.8421	337426.6	120000000
flow_iat_std	float64	2830743	0	1056642	2830743	2919270.92	8045869.78	0	0	137.178716	691266.282	84800261.6
flow_iat_max	int64	2830743	0	580289	2830743	9182475.32	24459539.3	-13	123	30865	2440145	120000000
flow_iat_min	int64	2830743	0	136316	2830743	162379.562	2950281.78	-14	3	4	64	120000000
fwd_iat_total	int64	2830743	0	493098	2830743	14482961.7	33575811.6	0	0	43	1242843.5	120000000
fwd_iat_mean	float64	2830743	0	737737	2830743	2610192.97	9525722.49	0	0	26	206306.358	120000000
fwd_iat_std	float64	2830743	0	700313	2830743	3266957.22	9639055.4	0	0	0	65989.8217	84602929.3
fwd_iat_max	int64	2830743	0	437316	2830743	9042938.56	24529157.4	0	0	43	931006	120000000
fwd_iat_min	int64	2830743	0	110631	2830743	1021892.91	8591436.34	-12	0	3	48	120000000
bwd_iat_total	int64	2830743	0	414928	2830743	9893830.35	28736614.2	0	0	3	98580.5	120000000
bwd_iat_mean	float64	2830743	0	670824	2830743	1805783.77	8887197.08	0	0	3	18248.5674	120000000
bwd_iat_std	float64	2830743	0	709042	2830743	1485973.35	6278468.57	0	0	0	15724.093	84418013.8
bwd_iat_max	int64	2830743	0	368285	2830743	4684692.43	17160950.1	0	0	3	60210	120000000
bwd_iat_min	int64	2830743	0	66074	2830743	967261.371	8308983.09	0	0	1	45	120000000
fwd_psh_flags	int64	2830743	0	2	2830743	0.04644646	0.21045001	0	0	0	0	1
bwd_psh_flags	int64	2830743	0	1	2830743	0	0	0	0	0	0	0
fwd_urg_flags	int64	2830743	0	2	2830743	0.00011128	0.01054826	0	0	0	0	1
bwd_urg_flags	int64	2830743	0	1	2830743	0	0	0	0	0	0	0
fwd_header_length	int64	2830743	0	3771	2830743	-25997.392	21052857.9	-3.221E+10	40	64	120	4644908
bwd_header_length	int64	2830743	0	3945	2830743	-2273.2753	1452208.94	-1.074E+09	20	40	104	5838440
fwd_packets/s	float64	2830743	0	1220423	2830743	63865.3464	247537.13	0	1.74944645	61.3252384	12048.1928	3000000
bwd_packets/s	float64	2830743	0	1107886	2830743	6995.19174	38151.6977	0	0.12291971	19.8278939	7352.94118	2000000
min_packet_length	int64	2830743	0	215	2830743	16.4345047	25.2377208	0	0	2	36	1448
max_packet_length	int64	2830743	0	5708	2830743	950.402445	2028.22915	0	6	87	525	24820
packet_length_mean	float64	2830743	0	215826	2830743	171.94439	305.491461	0	6	57.2	119.8	3337.14286
packet_length_std	float64	2830743	0	412246	2830743	294.975554	631.800073	0	0	25.9807621	174.323949	4731.52239
packet_length_variance	float64	2830743	0	405565	2830743	486154.794	1647489.87	0	0	675	30388.8393	22400000
fin_flag_count	int64	2830743	0	2	2830743	0.03537976	0.18473777	0	0	0	0	1
syn_flag_count	int64	2830743	0	2	2830743	0.04644646	0.21045001	0	0	0	0	1
rst_flag_count	int64	2830743	0	2	2830743	0.00024234	0.01556536	0	0	0	0	1
psh_flag_count	int64	2830743	0	2	2830743	0.29807051	0.45741071	0	0	0	1	1
ack_flag_count	int64	2830743	0	2	2830743	0.31584429	0.46485132	0	0	0	1	1
urg_flag_count	int64	2830743	0	2	2830743	0.09482316	0.29297058	0	0	0	0	1
cwe_flag_count	int64	2830743	0	2	2830743	0.00011128	0.01054826	0	0	0	0	1
ece_flag_count	int64	2830743	0	2	2830743	0.0002434	0.01559935	0	0	0	0	1
down/up_ratio	int64	2830743	0	31	2830743	0.68350041	0.68049199	0	0	1	1	156
average_packet_size	float64	2830743	0	212207	2830743	191.983658	331.860349	0	7.5	72.25	149.263932	3893.33333
avg_fwd_segment_size	float64	2830743	0	99716	2830743	58.2019385	186.091157	0	6	34	50	5940.85714
avg_bwd_segment_size	float64	2830743	0	147611	2830743	305.949281	605.2568	0	0	72	181	5800.5
fwd_header_length.1	int64	2830743	0	3771	2830743	-25997.392	21052857.9	-3.221E+10	40	64	120	4644908
fwd_avg_bytes/bulk	int64	2830743	0	1	2830743	0	0	0	0	0	0	0
fwd_avg_packets/bulk	int64	2830743	0	1	2830743	0	0	0	0	0	0	0
fwd_avg_bulk_rate	int64	2830743	0	1	2830743	0	0	0	0	0	0	0
bwd_avg_bytes/bulk	int64	2830743	0	1	2830743	0	0	0	0	0	0	0
bwd_avg_packets/bulk	int64	2830743	0	1	2830743	0	0	0	0	0	0	0
bwd_avg_bulk_rate	int64	2830743	0	1	2830743	0	0	0	0	0	0	0
subflow_fwd_packets	int64	2830743	0	1432	2830743	9.36115995	749.672792	1	2	2	5	219759
subflow_fwd_bytes	int64	2830743	0	17928	2830743	549.291936	9980.07024	0	12	62	187	12870338
subflow_bwd_packets	int64	2830743	0	1747	2830743	10.3937701	997.388269	0	1	2	4	291922
subflow_bwd_bytes	int64	2830743	0	64738	2830743	16162.3025	2263057.28	0	0	123	482	655453030
init_win_bytes_forward	int64	2830743	0	12151	2830743	6989.83745	14338.7258	-1	-1	251	8192	65535
init_win_bytes_backward	int64	2830743	0	13112	2830743	1989.43298	8456.88316	-1	-1	-1	235	65535
act_data_pkt_fwd	int64	2830743	0	1093	2830743	5.41821776	636.425664	0	0	1	2	213557
min_seg_size_forward	int64	2830743	0	28	2830743	-2741.6879	1084989.38	-536870661	20	24	32	138
active_mean	float64	2830743	0	326325	2830743	81551.321	648599.937	0	0	0	0	110000000
active_std	float64	2830743	0	202826	2830743	41134.1168	393381.515	0	0	0	0	74200000
active_max	float64	2830743	0	299565	2830743	153182.521	1025824.97	0	0	0	0	110000000
active_min	int64	2830743	0	175670	2830743	58295.8177	577092.282	0	0	0	0	110000000
idle_mean	float64	2830743	0	222016	2830743	8316036.63	23630078.6	0	0	0	0	120000000
idle_std	float64	2830743	0	197616	2830743	503843.95	4602984.48	0	0	0	0	76900000
idle_max	int64	2830743	0	149737	2830743	8695751.98	24366888.3	0	0	0	0	120000000
idle_min	int64	2830743	0	223888	2830743	7920031.01	23363418.9	0	0	0	0	120000000

Anexo E: Tabla de Resumen Después de la Limpieza de Datos

	tipo_dato	no_nulos	nulos	valores_unicos	count	mean	std	min	25%	50%	75%	max
flow_bytes/s	float64	2603842	949	1591480	2603842	9845832.739	745.152184	0	115.6653062	3778.272603	122448.9796	inf
flow_iat_min	float64	2604597	194	136297	2604597	162225.1773	2940169.989	0	3	4	63	120000000
fwd_iat_min	float64	2604774	17	110603	2604774	1096493.297	8906803.163	0	0	3	48	120000000
destination_port	int64	2604791	0	53800	2604791	8526.501771	18742.97417	0	53	80	443	65535
total_backward_packets	int64	2604791	0	1393	2604791	5.957699869	107.2735587	0	1	2	5	157388
total_fwd_packets	int64	2604791	0	1105	2604791	6.051365349	77.54137362	1	1	2	5	112141
flow_duration	int64	2604791	0	1048660	2604791	15995932.78	34725769.72	0	185	48104	5138656.5	119999998
total_length_of_fwd_packets	int64	2604791	0	17557	2604791	568.3856025	9551.830143	0	12	64	318	12900000
fwd_packet_length_mean	float64	2604791	0	98332	2604791	61.48461334	193.2673342	0	6	35	51.33333333	5940.857143
fwd_packet_length_std	float64	2604791	0	251966	2604791	74.61357609	292.1033677	0	0	0	69.5139874	7125.596846
fwd_packet_length_max	int64	2604791	0	5263	2604791	223.0643622	744.186387	0	6	38	196	24820
total_length_of_bwd_packets	int64	2604791	0	63305	2604791	5807.307348	261090.1515	0	6	142	796	399000000
bwd_packet_length_min	int64	2604791	0	581	2604791	41.91369365	70.06944837	0	0	0	79	2896
bwd_packet_length_max	int64	2604791	0	4811	2604791	940.4843225	2010.296467	0	6	91	509	19530
bwd_packet_length_std	float64	2604791	0	246864	2604791	363.7240736	869.2267494	0	0	0	199.1030476	8194.660487
bwd_packet_length_mean	float64	2604791	0	145861	2604791	328.5842979	624.1309329	0	6	82	207	5800.5
flow_packets/s	float64	2604791	0	1237758	2604791	9946834.739	25321845.02	0.016701329	2.402734529	82.43173622	20618.5567	253218459
flow_iat_mean	float64	2604791	0	1163904	2604791	1396267.17	4598706.031	0	72	17024.33333	466401.4203	120000000
flow_iat_std	float64	2604791	0	1054261	2604791	3170350.382	8339486.683	0	0	7957.340573	1353140.203	84800261.57
fwd_packet_length_min	int64	2604791	0	384	2604791	18.7293852	62.41899708	0	0	2	36	2325
flow_iat_max	int64	2604791	0	579532	2604791	9946833.739	25321845.02	0	149	36610	4788586.5	120000000
fwd_iat_total	int64	2604791	0	492151	2604791	15668699.28	34655976.73	0	0	48	3992909	120000000
fwd_iat_std	float64	2604791	0	698015	2604791	3547367.369	9997821.486	0	0	0	197651.8639	84602929.28
fwd_iat_mean	float64	2604791	0	735381	2604791	2821435.82	9860586.622	0	0	48	648890.4375	120000000
fwd_iat_max	int64	2604791	0	436530	2604791	9796286.229	25399757.22	0	0	48	3178825	120000000
bwd_iat_total	int64	2604791	0	414111	2604791	1069378.18	29730160.04	0	0	3	145648	120000000
bwd_iat_mean	float64	2604791	0	668529	2604791	1961408.183	9247960.85	0	0	3	27629.375	120000000
bwd_iat_std	float64	2604791	0	706761	2604791	1612365.541	6527915.01	0	0	0	23530.56862	84418013.78
bwd_iat_max	int64	2604791	0	367533	2604791	5075318.837	17815912.85	0	0	3	96672	120000000
bwd_iat_min	int64	2604791	0	66036	2604791	1051144.242	8656775.688	0	0	2	46	120000000
fwd_psh_flags	int64	2604791	0	2	2604791	0.046268587	0.210066233	0	0	0	0	1
fwd_urg_flags	int64	2604791	0	2	2604791	3.07126E-05	0.005541814	0	0	0	0	1
fwd_header_length	int64	2604791	0	3234	2604791	159.8149318	1678.902568	0	40	64	136	2289684
bwd_header_length	int64	2604791	0	3448	2604791	157.5571549	2207.145118	0	20	40	112	3147772
fwd_packets/s	float64	2604791	0	1218030	2604791	54073.62999	231665.8913	0	1.365409348	41.78854994	10416.66667	3000000
bwd_packets/s	float64	2604791	0	1105498	2604791	6694.291491	36215.54331	0	0.166154689	20.34919214	5571.030641	2000000
min_packet_length	int64	2604791	0	215	2604791	16.26728286	25.37304813	0	0	2	36	1448
max_packet_length	int64	2604791	0	5692	2604791	1026.292147	2093.371519	0	6	98	911	24820
packet_length_mean	float64	2604791	0	213984	2604791	183.8673911	314.4163342	0	6	61.33333333	140.5952381	3337.142857
packet_length_std	float64	2604791	0	410117	2604791	318.855353	652.4892615	0	2.309401077	30.67246322	278.7364048	4731.522394
packet_length_variance	float64	2604791	0	403439	2604791	527381.4789	1710754.566	0	5.333333333	940.8	77693.98333	22400000
fin_flag_count	int64	2604791	0	2	2604791	0.030957186	0.173201762	0	0	0	0	1
syn_flag_count	int64	2604791	0	2	2604791	0.046268587	0.210066233	0	0	0	0	1
rst_flag_count	int64	2604791	0	2	2604791	0.000261057	0.016155165	0	0	0	0	1
psh_flag_count	int64	2604791	0	2	2604791	0.313381381	0.463868056	0	0	0	1	1
ack_flag_count	int64	2604791	0	2	2604791	0.308747228	0.461976687	0	0	0	1	1
urg_flag_count	int64	2604791	0	2	2604791	0.09625993	0.294947435	0	0	0	0	1
cwe_flag_count	int64	2604791	0	2	2604791	3.07126E-05	0.005541814	0	0	0	0	1
ece_flag_count	int64	2604791	0	2	2604791	0.000262209	0.016190753	0	0	0	0	1
down/up_ratio	int64	2604791	0	31	2604791	0.6999456	0.688513471	0	0	1	1	156
average_packet_size	float64	2604791	0	210376	2604791	204.9834611	341.6561271	0	8	77.5	167.25	3893.333333
avg_fwd_segment_size	float64	2604791	0	98332	2604791	61.48461334	193.2673342	0	6	35	51.33333333	5940.857143
avg_bwd_segment_size	float64	2604791	0	145859	2604791	328.5842979	624.1309329	0	6	82	207	5800.5
subflow_fwd_packets	int64	2604791	0	1105	2604791	6.051365349	77.54137362	1	1	2	5	112141
subflow_fwd_bytes	int64	2604791	0	17557	2604791	568.3742151	9536.457034	0	12	64	318	12870338
subflow_bwd_packets	int64	2604791	0	1393	2604791	5.957699869	107.2735587	0	1	2	5	157388
subflow_bwd_bytes	int64	2604791	0	63310	2604791	5807.401864	261303.5323	0	6	142	796	399368696
act_data_pkt_fwd	int64	2604791	0	854	2604791	3.037578447	70.8564372	0	0	1	3	108273
min_seg_size_forward	int64	2604791	0	24	2604791	26.20990782	6.63719822	0	20	24	32	138
active_mean	float64	2604791	0	324832	2604791	87361.64316	656056.085	0	0	0	0	110000000
active_std	float64	2604791	0	201671	2604791	44136.62387	403634.2644	0	0	0	0	74200000
active_max	int64	2604791	0	298199	2604791	164293.9029	1049098.096	0	0	0	0	110000000
active_min	int64	2604791	0	175036	2604791	62407.22645	581701.8876	0	0	0	0	110000000
idle_mean	float64	2604791	0	221303	2604791	9020411.551	24490869.29	0	0	0	0	120000000
idle_std	float64	2604791	0	196506	2604791	546530.9655	4793361.751	0	0	0	0	76900000
idle_max	int64	2604791	0	149411	2604791	9432290.31	25250536.78	0	0	0	0	120000000
idle_min	int64	2604791	0	223244	2604791	8591013.959	24223850.74	0	0	0	0	120000000

Anexo F: Distribuciones por característica antes y después de imputaciones por regresores bayesianos y de tipo árbol de decisión

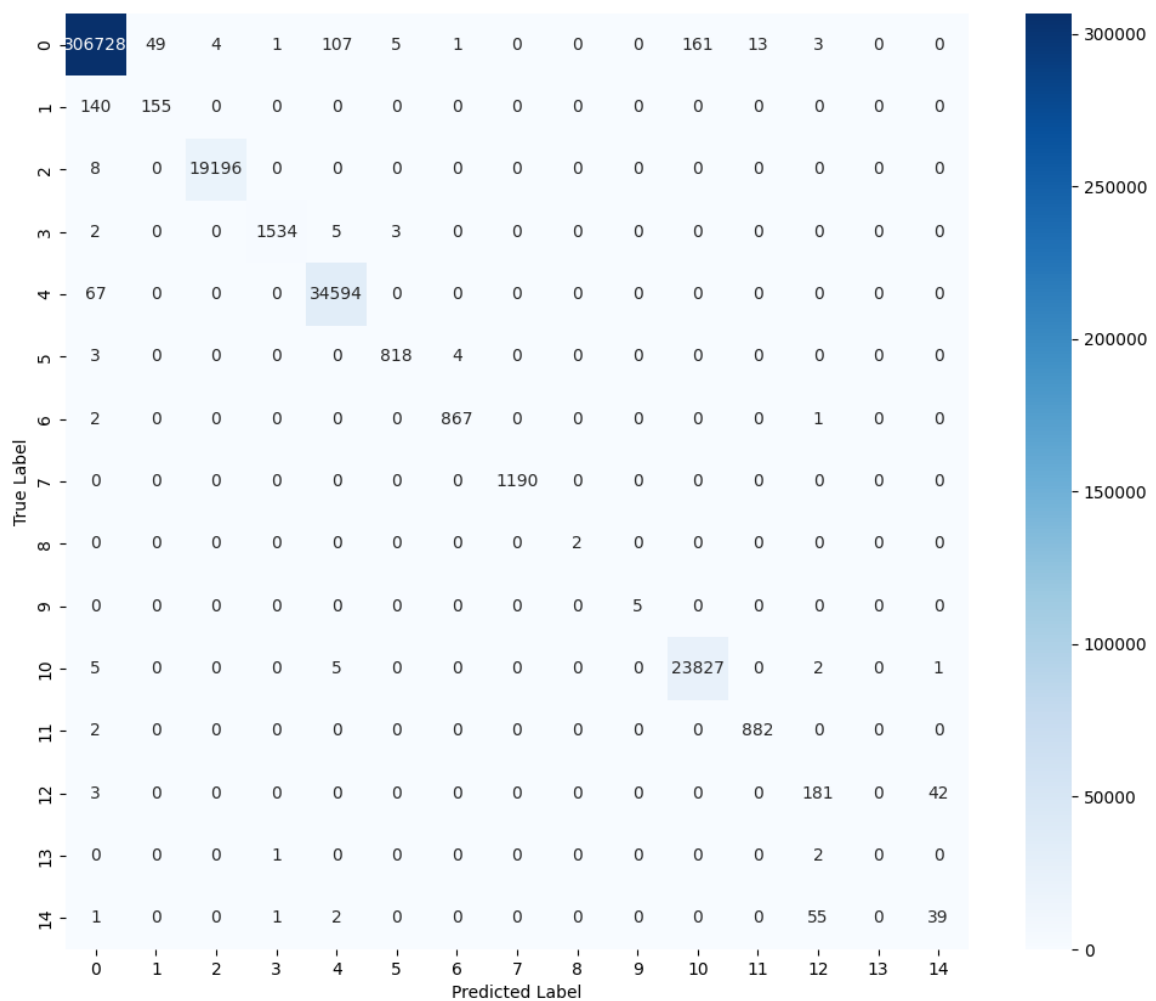




Anexo G: Matrices de Confusión de Modelos Base

Descripción de Labels de la Matriz: Tráfico Benigno: 0, Ataque Bot: 1, DDoS: 2, DoS GoldenEye: 3, DoS Hulk: 4, DoS Slowhttptest: 5, DoS Slowloris: 6, FTP- Patator: 7, Heartbleed: 8, Infiltration: 9, PortScan: 10, SSH- Patator: 11, Web Attack- Brute Force: 12, Web Attack- SQL Injection: 13, Web Attack- XSS: 14

• Random Forest



Detalle de Matriz: La matriz indica a nivel de filas los valores reales y a nivel de columnas valores predichos. Por ende, se presentan cantidad de predicciones respecto a valores reales

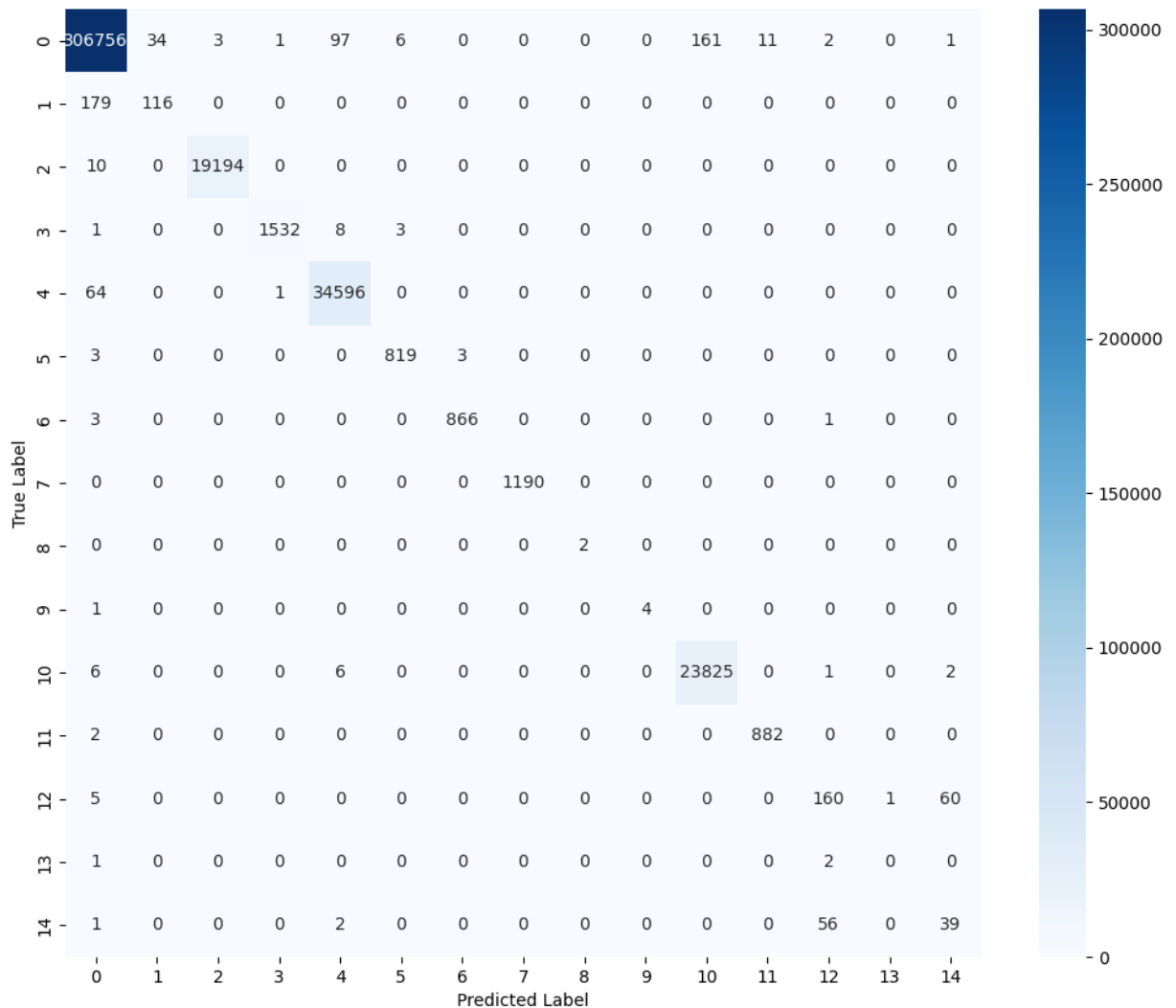
Primera Columna: Falsos Negativos

Primera Fila: Falsos Positivos

Diagonal: Predicciones Correctas

Casillas Restantes: Predicciones Incorrectas

- **XGBoost**



Detalle de Matriz: La matriz indica a nivel de filas los valores reales y a nivel de columnas valores predichos. Por ende, se presentan cantidad de predicciones respecto a valores reales

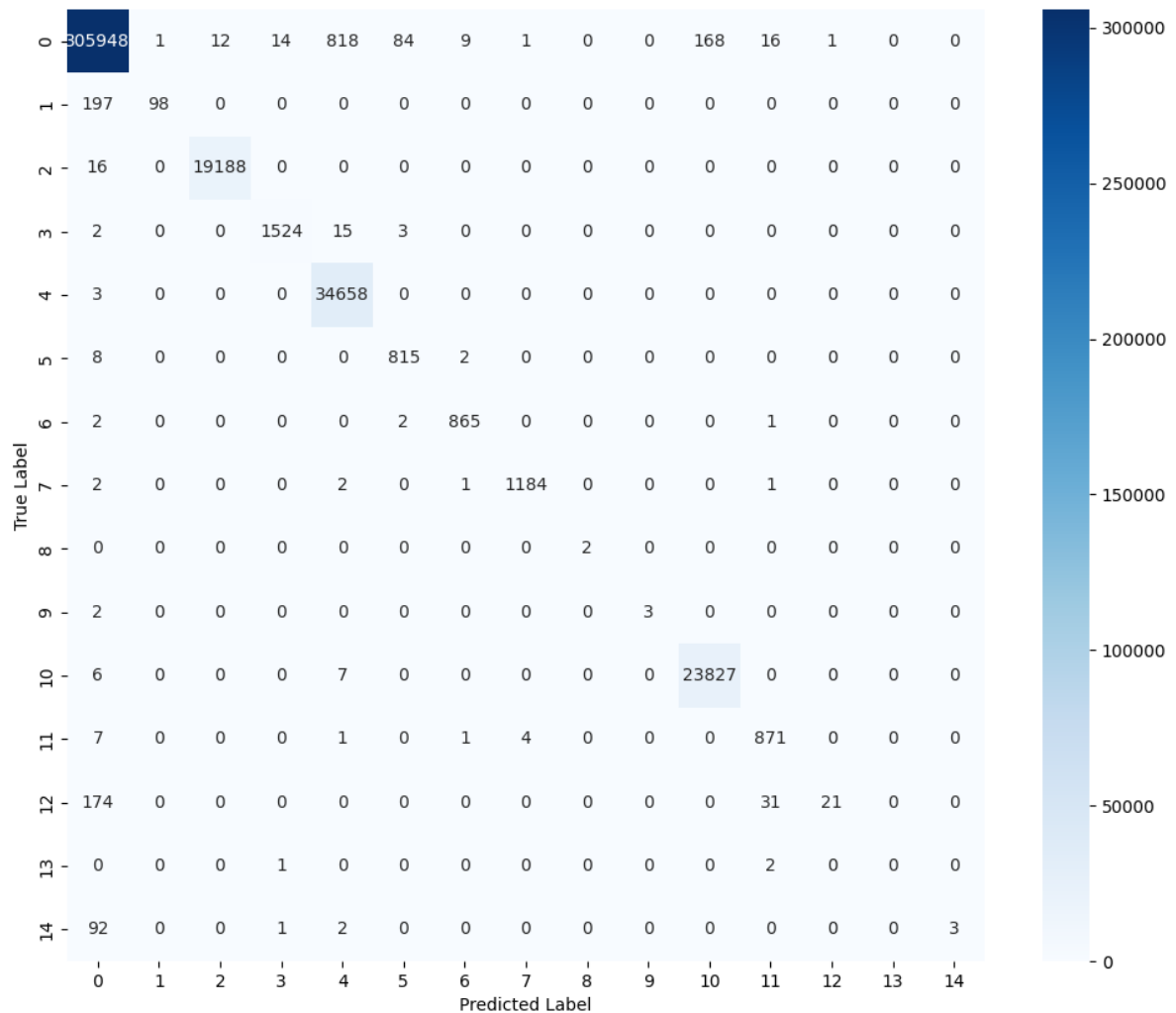
Primera Columna: Falsos Negativos

Primera Fila: Falsos Positivos

Diagonal: Predicciones Correctas

Casillas Restantes: Predicciones Incorrectas

- Redes Neuronales tipo Perceptrón Multicapa**



Detalle de Matriz: La matriz indica a nivel de filas los valores reales y a nivel de columnas valores predichos. Por ende, se presentan cantidad de predicciones respecto a valores reales

Primera Columna: Falsos Negativos

Primera Fila: Falsos Positivos

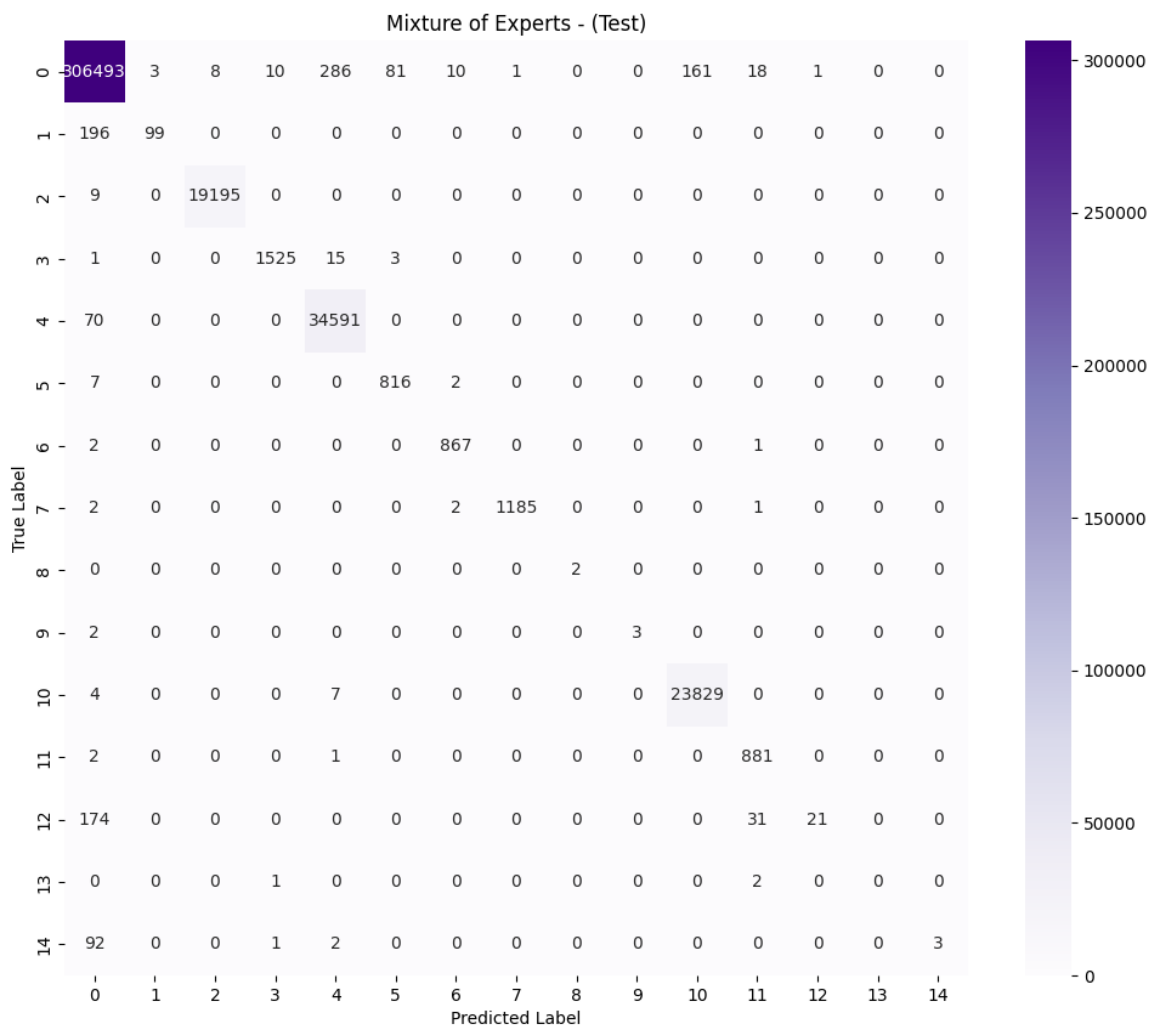
Diagonal: Predicciones Correctas

Casillas Restantes: Predicciones Incorrectas

Anexo H. Matrices de Confusión de Arquitecturas Mixture of Experts

Descripción de Labels de la Matriz: Tráfico Benigno: 0, Ataque Bot: 1, DDoS: 2, DoS GoldenEye: 3, DoS Hulk: 4, DoS Slowhttptest: 5, DoS Slowloris: 6, FTP- Patator: 7, Heartbleed: 8, Infiltration: 9, PortScan: 10, SSH- Patator: 11, Web Attack- Brute Force: 12, Web Attack- SQL Injection: 13, Web Attack- XSS: 14

- Matriz de Confusión- Primera Arquitectura:**



Detalle de Matriz: La matriz indica a nivel de filas los valores reales y a nivel de columnas valores predichos. Por ende, se presentan cantidad de predicciones respecto a valores reales

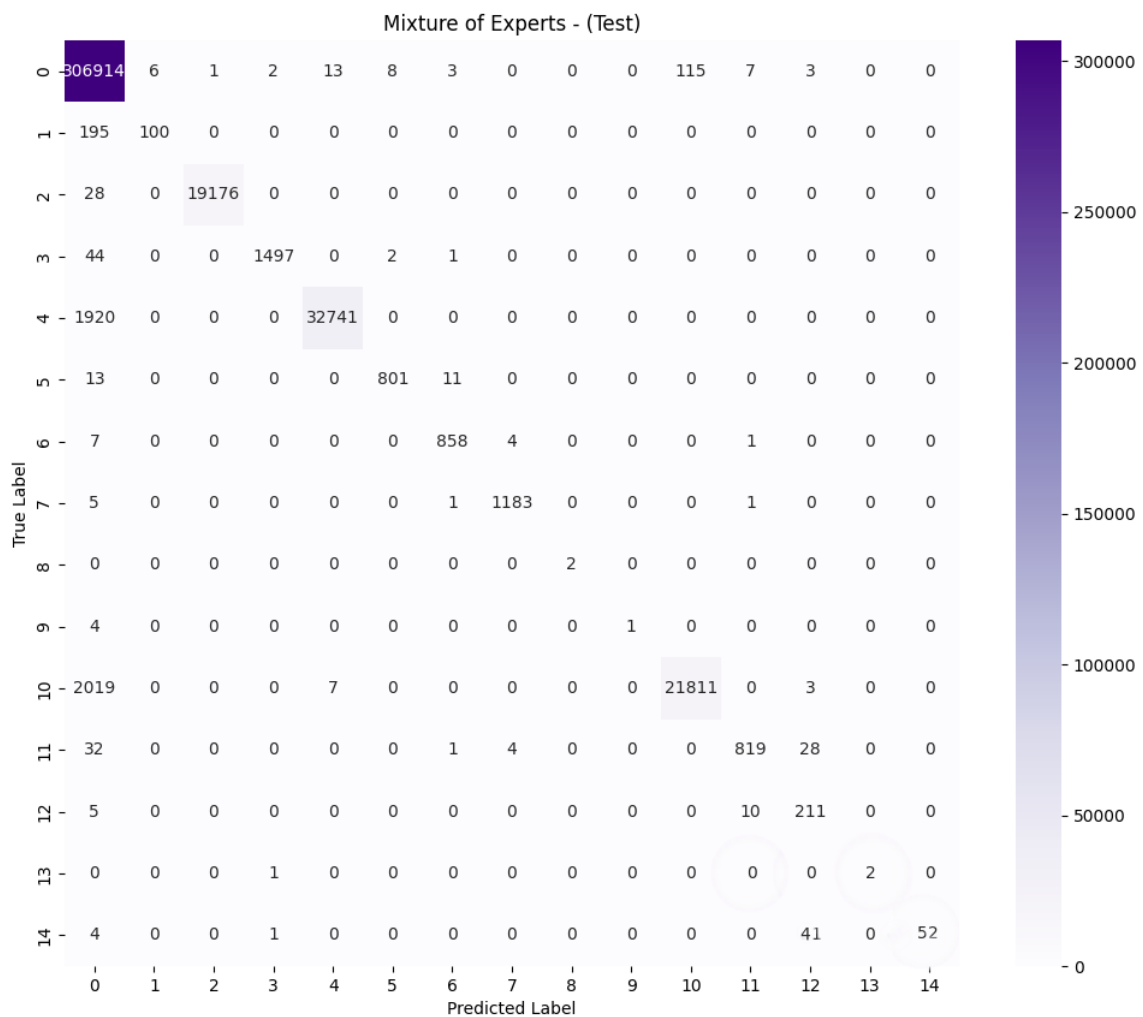
Primera Columna: Falsos Negativos

Primera Fila: Falsos Positivos

Diagonal: Predicciones Correctas

Casillas Restantes: Predicciones Incorrectas

- Matriz de Confusión- Segunda Arquitectura:**



Detalle de Matriz: La matriz indica a nivel de filas los valores reales y a nivel de columnas valores predichos. Por ende, se presentan cantidad de predicciones respecto a valores reales

Primera Columna: Falsos Negativos

Primera Fila: Falsos Positivos

Diagonal: Predicciones Correctas

Casillas Restantes: Predicciones Incorrectas

- Matriz de Confusión- Tercera Arquitectura:**



Detalle de Matriz: La matriz indica a nivel de filas los valores reales y a nivel de columnas valores predichos. Por ende, se presentan cantidad de predicciones respecto a valores reales

Primera Columna: Falsos Negativos

Primera Fila: Falsos Positivos

Diagonal: Predicciones Correctas

Casillas Restantes: Predicciones Incorrectas